

Thirty characters are not enough

CPC Reference Notebook

Table of Contents

1	Template and Stress	2	HLD.cpp	15
	Template.cpp	2	Hopcroftkarp.cpp	15
	stress.bat	2	LCA and Binary lifting.cpp	16
	brute.cpp	2	MCMF.cpp	16
	fast.cpp	2	Rerooting.cpp	16
	generator.cpp	2	SCC.cpp	17
2	Data structures	4	tree isomorphism.cpp	17
	bin_Trie.cpp	4	Virtual Tree.cpp	17
	CHT.cpp	4	4 Math temps	19
	DSU rollbacks.cpp	4	modnum helper	19
	DSU.cpp	4	BerlekampMassey.cpp	19
	Fenwick.cpp	5	comb.cpp	19
	lichao segment with rollbacks.cpp	5	CRT.cpp	19
	lichao with rollbacks.cpp	5	Discrete log.cpp	20
	merge sort tree.cpp	6	FFT string_matching.cpp	20
	Mo_s with updates.cpp	6	FFT with MOD.cpp	20
	Mo_s_algorithm.cpp	6	FFT.cpp	21
	Multiset lazy deletion.cpp	6	FWHT.cpp	21
	Offline 2D BIT.cpp	7	Gaussian with Doubles.cpp	22
	ordered_set.cpp	7	Gaussian with MOD.cpp	22
	Persistent DSU.cpp	7	lagrange interpolation.cpp	22
	PST.cpp	7	matrix_expo.cpp	23
	seg_tree (lazy generic).cpp	8	NTT.cpp	23
	seg_tree (lazy propagation).cpp	8	NTT_CRT.cpp	24
	seg_tree (point, range query).cpp	8	Number theory.cpp	25
	sparse_table.cpp	9	Pollard Rho Miller Rabin.cpp	25
	sqrt_decomposition.cpp	9	Primitive root.cpp	26
	topK.cpp	9	rand.cpp	26
	TwoStackQ.cpp	9	Segmented sieve.cpp	26
3	graphs	11	SOS dp.cpp	26
	2SAT.cpp	11	xor basis.cpp	27
	Articulation points.cpp	11	5 strings	29
	Biconnected components.cpp	11	Aho.cpp	29
	Bipartite check.cpp	12	dynamic_aho.cpp	29
	Blossom.cpp	12	Generalized suffix automaton.cpp	29
	Bridges.cpp	13	KMP.cpp	30
	Centroid decomposition buckets.cpp	13	manacher.cpp	31
	Centroid decomposition with updates.cpp	13	palindromic tree.cpp	31
	Centroid decomposition.cpp	14	String Hashing.cpp	32
	dijkstra.cpp	14	Suffixautomaton.cpp	32
	Dinic.cpp	14	Trie_strings.cpp	33
	DSU on trees.cpp	15	Z_algorithm.cpp	33
	eulerWalk.cpp	15	6 DP Optimizations and Misc	34
			divide and conquer dp.cpp	34
			Custom hashing.cpp	34

1 Template and Stress

Template.cpp

```
#include <bits/stdc++.h>
#pragma GCC optimize("Ofast, unroll-loops")
#ifdef ONLINE_JUDGE
#define debug(...) ((void)0)
#else
#include "Template/debug.h"
#endif
using namespace std;
using ll = long long;
#define endl '\n'

void SOLVE(){
}

signed main(){
    ios_base::sync_with_stdio(false); cin.tie(nullptr);
    int o_o; cin >> o_o; while(o_o--){
        SOLVE(); return 0;
    }
}
```

stress.bat

```
@echo off
setlocal EnableDelayedExpansion

if not exist bin mkdir bin

g++ -std=c++17 -O2 generator.cpp -o bin/gen.exe || exit /b
g++ -std=c++17 -O2 brute.cpp -o bin/brute.exe || exit /b
g++ -std=c++17 -O2 fast.cpp -o bin/fast.exe || exit /b

set /a I=0
set LIM=%1
if "%LIM%"=="" set LIM=0

:loop
set /a I+=1
echo Test %I%

bin/gen.exe %I% > input.txt
bin/brute.exe < input.txt > out_brute.txt
bin/fast.exe < input.txt > out_fast.txt

fc out_brute.txt out_fast.txt > nul || (
    echo MISMATCH ON SEED %I%
    echo ==== INPUT ==== & type input.txt
    echo ==== BRUTE ==== & type out_brute.txt
    echo ==== FAST ==== & type out_fast.txt
    pause
    exit /b
)

if not "%LIM%"=="0" if %I% GEQ %LIM% exit /b
goto loop
```

brute.cpp

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

void solve() {
    int n;
    cin >> n;
    vector<i64> a(n);
    for (i64 &x : a) cin >> x;

    i64 best = LLONG_MIN;
    for (int l = 0; l < n; l++) {
        i64 sum = 0;
        for (int r = l; r < n; r++) {
            sum += a[r];
            best = max(best, sum);
        }
    }
    cout << best << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) solve();
}
```

fast.cpp

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

void solve() {
    int n;
    cin >> n;
    vector<i64> a(n);
    for (i64 &x : a) cin >> x;

    i64 best = a[0], current = a[0];
    for (int i = 1; i < n; i++) {
        current = max(a[i], current + a[i]);
        best = max(best, current);
    }
    cout << best << '\n';
}
```

```
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) solve();
}
```

generator.cpp

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

ll rnd(ll l, ll r) {
    return uniform_int_distribution<ll>(l, r)(rng);
}

int rnd(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}

bool coin(int p = 50) {
    return rnd(1, 100) <= p;
}

template <class T>
void print(vector<T> a) {
    for (int i = 0; i < (int)a.size(); i++) cout << a[i] << " \n"[i + 1 ==
        (int)a.size()];
}

vector<ll> arr(int n, ll l, ll r) {
    vector<ll> a(n);
    for (auto &x : a) x = rnd(l, r);
    return a;
}

vector<int> perm(int n) {
    vector<int> p(n);
    iota(p.begin(), p.end(), 1);
    shuffle(p.begin(), p.end(), rng);
    return p;
}

string str(int n, char l = 'a', char r = 'z') {
    string s;
    while (n--) s += char(rnd(l, r));
    return s;
}

vector<string> grid(int n, int m, string chars = ".#") {
    vector<string> g(n, string(m, '.'));
    for (auto &row : g) {
        for (char &c : row) c = chars[rnd(0, (int)chars.size() - 1)];
    }
    return g;
}

vector<pair<int, int>> tree(int n) {
    vector<pair<int, int>> e;
    for (int i = 2; i <= n; i++) e.push_back({rnd(1, i - 1), i});
    shuffle(e.begin(), e.end(), rng);
    return e;
}

vector<pair<int, int>> path_tree(int n) {
    vector<pair<int, int>> e;
    for (int i = 2; i <= n; i++) e.push_back({i - 1, i});
    return e;
}

vector<pair<int, int>> star_tree(int n) {
    vector<pair<int, int>> e;
    int c = rnd(1, n);
    for (int i = 1; i <= n; i++) {
        if (i != c) e.push_back({c, i});
    }
    shuffle(e.begin(), e.end(), rng);
    return e;
}

vector<pair<int, int>> graph(int n, int m) {
    vector<pair<int, int>> all, e;
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) all.push_back({i, j});
    }
    shuffle(all.begin(), all.end(), rng);
    m = min(m, (int)all.size());
    for (int i = 0; i < m; i++) e.push_back(all[i]);
    return e;
}

vector<pair<int, int>> digraph(int n, int m) {
    vector<pair<int, int>> all, e;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i != j) all.push_back({i, j});
        }
    }
    shuffle(all.begin(), all.end(), rng);
    m = min(m, (int)all.size());
    for (int i = 0; i < m; i++) e.push_back(all[i]);
    return e;
}

vector<pair<int, int>> dag(int n, int m) {
    vector<int> p = perm(n);
    vector<pair<int, int>> all, e;
    for (int i = 0; i < n; i++) {
```

```

        for (int j = i + 1; j < n; j++) all.push_back({p[i], p[j]});
    }
    shuffle(all.begin(), all.end(), rng);
    m = min(m, (int)all.size());
    for (int i = 0; i < m; i++) e.push_back(all[i]);
    return e;
}

vector<pair<int, int>> connected_graph(int n, int m) {
    vector<pair<int, int>> e = tree(n);
    set<pair<int, int>> used;
    for (auto [u, v] : e) {
        if (u > v) swap(u, v);
        used.insert({u, v});
    }
    m = min(m, n * (n - 1) / 2);
    while ((int)e.size() < m) {
        int u = rnd(1, n), v = rnd(1, n);
        if (u == v) continue;
        if (u > v) swap(u, v);
        if (used.insert({u, v}).second) e.push_back({u, v});
    }
    shuffle(e.begin(), e.end(), rng);
    return e;
}

vector<pair<int, int>> queries(int q, int n) {
    vector<pair<int, int>> res;
    while (q--) {
        int l = rnd(1, n), r = rnd(1, n);
        if (l > r) swap(l, r);
        res.push_back({l, r});
    }
    return res;
}

void print_edges(vector<pair<int, int>> e) {
    for (auto [u, v] : e) cout << u << ' ' << v << '\n';
}

void print_weighted_edges(vector<pair<int, int>> e, int l, int r) {
    for (auto [u, v] : e) cout << u << ' ' << v << ' ' << rnd(l, r) << '\n';
}

void solve() {
    int n = rnd(1, 20);
    cout << n << '\n';
    print(arr(n, -20, 20));
}

int main(int argc, char **argv) {
    ios::sync_with_stdio(0);
    cin.tie(0);
    if (argc > 1) rng.seed(stoull(argv[1]));
    int t = 1; // TESTCASES
    if (argc > 2) t = stoi(argv[2]);
    cout << t << '\n';
    while (t--) solve();
}

```

2 Data structures

bin_Trie.cpp

```
struct Trie{
    struct Node{
        int child[2], freq;
        Node(){
            child[0] = child[1] = -1;
            freq = 0;
        }
    };
    vector<Node> trie;
    int bits;
    Trie(int x){
        bits = x;
        trie.emplace_back();
    }
    int newNode(){
        trie.emplace_back();
        return (int)trie.size() - 1;
    }
    void insert(ll val){
        int cur = 0;
        trie[cur].freq++;
        for (int i = bits; i >= 0; i--){
            bool bit = (val >> i) & 1;
            if (trie[cur].child[bit] == -1){
                trie[cur].child[bit] = newNode();
            }
            cur = trie[cur].child[bit];
            trie[cur].freq++;
        }
    }
    void del(ll val){
        if (!cnt(val)) return;
        int cur = 0;
        trie[cur].freq--;
        for (int i = bits; i >= 0; i--){
            bool bit = (val >> i) & 1;
            cur = trie[cur].child[bit];
            trie[cur].freq--;
        }
    }
    int cnt(ll val){
        int cur = 0;
        for (int i = bits; i >= 0; i--){
            bool bit = (val >> i) & 1;
            cur = trie[cur].child[bit];
            if (cur == -1) return 0;
        }
        return trie[cur].freq;
    }
    ll query(ll val, ll k){ // num ^ val < k
        int cur = 0;
        ll answer = 0;
        for (int i = bits; i >= 0 && cur != -1; i--){
            bool b1 = (val >> i) & 1;
            bool b2 = (k >> i) & 1;
            if (b2){
                int same = trie[cur].child[b1];
                if (same != -1) answer += trie[same].freq;
                cur = trie[cur].child[b1 ^ 1];
            }else{
                cur = trie[cur].child[b1];
            }
        }
        return answer;
    }
    ll mxXor(ll val){
        int cur = 0;
        ll ret = 0;
        for (int i = bits; i >= 0 && cur != -1; i--){
            bool bit = (val >> i) & 1;
            int nxt = trie[cur].child[bit ^ 1];
            if (nxt != -1 && trie[nxt].freq){
                ret |= (1LL << i);
                cur = nxt;
            }else{
                cur = trie[cur].child[bit];
            }
        }
        return ret;
    }
    ll mnXor(ll val){
        int cur = 0;
        ll ret = 0;
        for (int i = bits; i >= 0 && cur != -1; i--){
            bool bit = (val >> i) & 1;
            int nxt = trie[cur].child[bit];
            if (nxt != -1 && trie[nxt].freq){
                cur = nxt;
            }else{
                ret |= (1LL << i);
                cur = trie[cur].child[bit ^ 1];
            }
        }
        return ret;
    }
};

void SOLVE(){
}
```

CHT.cpp

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line &o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};
```

```
// CHT<true> for min, CHT<false> for max. add(k, m) adds y = k*x + m
template<bool is_min = false> struct CHT : multiset<Line, less<>> {
    // (for doubles, use inf = 1/0.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end())
            return x->p = inf, 0;
        if (x->k == y->k)
            x->p = x->m > y->m ? inf : -inf;
        else
            x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        if constexpr (is_min) {
            k = -k;
            m = -m;
        }
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z))
            z = erase(z);
        if (x != begin() && isect(--x, y))
            isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        if (empty())
            return is_min ? LLONG_MAX : LLONG_MIN;
        auto l = *lower_bound(x);
        ll res = l.k * x + l.m;
        if constexpr (is_min) {
            return -res;
        }
        return res;
    }
};

void SOLVE() {
}
```

DSU rollbacks.cpp

```
struct DSU{
    int n, components;
    vector<int> parent, sz;
    vector<pair<int&, int>> history;
    DSU(int n){
        init(n);
    }
    void init(int _n){
        n = _n;
        components = n;
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        history.clear();
        for (int i = 1; i <= n; i++){
            parent[i] = i;
        }
    }
    int find(int v){
        return parent[v] == v ? v : find(parent[v]);
    }
    bool same(int u, int v){
        return find(u) == find(v);
    }
    bool unite(int a, int b){
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        history.push_back({sz[a], sz[b]});
        history.push_back({parent[b], parent[b]});
        history.push_back({components, components});
        parent[b] = a;
        sz[a] += sz[b];
        components--;
        return true;
    }
    bool union_sets(int a, int b){
        return unite(a, b);
    }
    int time(){
        return (int)history.size();
    }
    int get_size(int v){
        return sz[find(v)];
    }
    void rollback(int until){
        while ((int)history.size() > until){
            history.back().first = history.back().second;
            history.pop_back();
        }
    }
};

void SOLVE(){
}
```

DSU.cpp

```
struct DSU{
    int n;
    vector<int> parent, size;
    int components;
    DSU(int n) : n(n), parent(n + 1), size(n + 1, 1), components(n) {
}
```

```

    iota(parent.begin(), parent.end(), 0);
}
int find(int v){
    return parent[v] = (parent[v] == v ? v : find(parent[v]));
}
bool same(int u, int v) {
    return find(u) == find(v);
}
bool union_sets(int a, int b){
    a = find(a);
    b = find(b);
    if (a == b) return false;
    if (size[a] < size[b]) swap(a, b);
    parent[b] = a;
    size[a] += size[b];
    components--;
    return true;
}
int get_size(int v){
    return size[find(v)];
}
}

void SOLVE() {
}

```

Fenwick.cpp

```

template <class T>
struct Fenwick { // 1-indexed
    int n, LOG; T ID;
    function<T(T, T)> op;
    vector<T> tree;
    Fenwick(int _n, T _ID, function<T(T, T)> _op) : n(_n + 5), ID(_ID), op(_op),
    LOG(log2(n) + 1) {
        tree.assign(n + 1, ID);
    }
    void add(int i, T val) {
        if (i <= 0) return;
        for (; i <= n; i += (i & -i)) tree[i] = op(tree[i], val);
    }
    T query(int i) {
        T res = ID;
        for (; i >= 1; i -= (i & -i)) res = op(res, tree[i]);
        return res;
    }
    T query(int l, int r){
        return query(r) - query(l - 1);
        // return query(r) ^ query(l - 1);
    }
    int lower_bound(ll val){
        T sum = 0;
        int pos = 0;
        for(int i = LOG; i >= 0; i--){
            if(pos + (1 << i) <= n && sum + tree[pos + (1 << i)] < val){
                pos += (1 << i);
                sum += tree[pos];
            }
        }
        return pos + 1;
    }
};

void SOLVE() {
}

```

lichao segment with rollbacks.cpp

```

template<bool is_min = true>
struct LiChaoSegment{
    const ll INF = 4e18;
    struct Line{
        ll a, b;
        Line(ll _a = 0, ll _b = 4e18) : a(_a), b(_b){}
    };
    vector<Line> tree;
    vector<pair<int, Line>> history;
    ll L, R;
    LiChaoSegment(){}
    LiChaoSegment(ll _L, ll _R){
        init(_L, _R);
    }
    void init(ll _L, ll _R){
        L = _L;
        R = _R;
        tree.assign(4, Line());
        history.clear();
    }
    ll eval(Line line, ll x){
        __int128 ret = (__int128)line.a * x + line.b;
        if (ret > INF) return INF;
        if (ret < -INF) return -INF;
        return (ll)ret;
    }
    bool better(Line x, Line y, ll at){
        return eval(x, at) < eval(y, at);
    }
    void save(int node){
        if (node >= (int)tree.size()) tree.resize(node + 1, Line());
        history.push_back({node, tree[node]});
    }
    void add_line(ll a, ll b){
        if constexpr (!is_min) a = -a, b = -b;
        add_line(Line(a, b), 1, L, R);
    }
    void add_line(Line cur, int node, ll l, ll r){
        if (node >= (int)tree.size()) tree.resize(node + 1, Line());
        ll mid = l + (r - l) / 2;
        bool lef = better(cur, tree[node], l);
        bool m = better(cur, tree[node], mid);
        bool r = better(cur, tree[node], r);
    }
}

```

```

    if (m){
        save(node);
        swap(cur, tree[node]);
    }
    if (l == r) return;
    if (lef != m) add_line(cur, node * 2, l, mid);
    else add_line(cur, node * 2 + 1, mid + 1, r);
}
void add_segment(ll a, ll b, ll ql, ll qr){
    if constexpr (!is_min) a = -a, b = -b;
    add_segment(Line(a, b), ql, qr, 1, L, R);
}
void add_segment(Line line, ll ql, ll qr, int node, ll l, ll r){
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr){
        add_line(line, node, l, r);
        return;
    }
    if (node >= (int)tree.size()) tree.resize(node + 1, Line());
    ll mid = l + (r - l) / 2;
    add_segment(line, ql, qr, node * 2, l, mid);
    add_segment(line, ql, qr, node * 2 + 1, mid + 1, r);
}
ll query(ll x){
    ll ret = query(x, 1, L, R);
    if constexpr (!is_min) ret = -ret;
    return ret;
}
ll query(ll x, int node, ll l, ll r){
    if (node >= (int)tree.size()) return INF;
    ll ret = eval(tree[node], x);
    if (l == r) return ret;
    ll mid = l + (r - l) / 2;
    if (x <= mid) ret = min(ret, query(x, node * 2, l, mid));
    else ret = min(ret, query(x, node * 2 + 1, mid + 1, r));
    return ret;
}
int time(){
    return (int)history.size();
}
void rollback(int t){
    while ((int)history.size() > t){
        auto [node, line] = history.back();
        history.pop_back();
        tree[node] = line;
    }
}

void SOLVE(){
}

```

lichao with rollbacks.cpp

```

template<bool is_min = true>
struct LiChao{
    const ll INF = 4e18;
    struct Line{
        ll a, b;
        Line(ll _a = 0, ll _b = 4e18) : a(_a), b(_b){}
    };
    vector<Line> tree;
    vector<pair<int, Line>> history;
    ll L, R;
    LiChao(){}
    LiChao(ll _L, ll _R){
        init(_L, _R);
    }
    void init(ll _L, ll _R){
        L = _L;
        R = _R;
        tree.assign(4, Line());
        history.clear();
    }
    ll eval(Line line, ll x){
        __int128 ret = (__int128)line.a * x + line.b;
        if (ret > INF) return INF;
        if (ret < -INF) return -INF;
        return (ll)ret;
    }
    bool better(Line x, Line y, ll at){
        return eval(x, at) < eval(y, at);
    }
    void save(int node){
        if (node >= (int)tree.size()) tree.resize(node + 1, Line());
        history.push_back({node, tree[node]});
    }
    void add_line(ll a, ll b){
        if constexpr (!is_min) a = -a, b = -b;
        add_line(Line(a, b), 1, L, R);
    }
    void add_line(Line cur, int node, ll l, ll r){
        if (node >= (int)tree.size()) tree.resize(node + 1, Line());
        ll mid = l + (r - l) / 2;
        bool lef = better(cur, tree[node], l);
        bool m = better(cur, tree[node], mid);
        if (m){
            save(node);
            swap(cur, tree[node]);
        }
        if (l == r) return;
        if (lef != m) add_line(cur, node * 2, l, mid);
        else add_line(cur, node * 2 + 1, mid + 1, r);
    }
    ll query(ll x){
        ll ret = query(x, 1, L, R);
        if constexpr (!is_min) ret = -ret;
        return ret;
    }
    ll query(ll x, int node, ll l, ll r){
        if (node >= (int)tree.size()) return INF;
        ll ret = eval(tree[node], x);
    }
}

```

```

    if (l == r) return ret;
    ll mid = 1 + (r - 1) / 2;
    if (x <= mid) ret = min(ret, query(x, node * 2, 1, mid));
    else ret = min(ret, query(x, node * 2 + 1, mid + 1, r));
    return ret;
}
int snapshot(){
    return (int)history.size();
}
}
void rollback(int t){
    while ((int)history.size() > t){
        auto [node, line] = history.back();
        history.pop_back();
        tree[node] = line;
    }
}
};

void SOLVE(){
}

```

merge sort tree.cpp

```

struct merge_sort_tree{
    int n;
    vector<vector<ll>> tree, pref;
    merge_sort_tree(){
        merge_sort_tree(vector<ll> &a){
            init(a);
        }
        void init(vector<ll> &a){
            n = 1;
            while (n < (int)a.size()) n <<= 1;
            tree.assign(n << 1, {});
            pref.assign(n << 1, {});
            build(0, 0, n, a);
        }
        void pull(int node){
            auto &l = tree[node * 2 + 1];
            auto &r = tree[node * 2 + 2];
            tree[node].resize(l.size() + r.size());
            merge(l.begin(), l.end(), r.begin(), r.end(), tree[node].begin());
            pref[node].resize(tree[node].size() + 1);
            for (int i = 0; i < (int)tree[node].size(); i++){
                pref[node][i + 1] = pref[node][i] + tree[node][i];
            }
        }
        void build(int node, int l, int r, vector<ll> &a){
            if (r - l == 1){
                if (l < (int)a.size()){
                    tree[node] = {a[l]};
                    pref[node] = {0, a[l]};
                }
                return;
            }
            int mid = (l + r) >> 1;
            build(node * 2 + 1, l, mid, a);
            build(node * 2 + 2, mid, r, a);
            pull(node);
        }
        int cnt_less(int node, ll x){
            return lower_bound(tree[node].begin(), tree[node].end(), x) -
                tree[node].begin();
        }
        int cnt_leq(int node, ll x){
            return upper_bound(tree[node].begin(), tree[node].end(), x) -
                tree[node].begin();
        }
        ll sum_first(int node, int cnt){
            return pref[node][cnt];
        }
        int count_less(int l, int r, ll x){
            return count_less(l, r + 1, x, 0, 0, n);
        }
        int count_less(int ql, int qr, ll x, int node, int l, int r){
            if (qr <= l || r <= ql) return 0;
            if (ql <= l && r <= qr) return cnt_less(node, x);
            int mid = (l + r) >> 1;
            return count_less(ql, qr, x, node * 2 + 1, l, mid) + count_less(ql, qr,
                x, node * 2 + 2, mid, r);
        }
        int count_leq(int l, int r, ll x){
            return count_leq(l, r + 1, x, 0, 0, n);
        }
        int count_leq(int ql, int qr, ll x, int node, int l, int r){
            if (qr <= l || r <= ql) return 0;
            if (ql <= l && r <= qr) return cnt_leq(node, x);
            int mid = (l + r) >> 1;
            return count_leq(ql, qr, x, node * 2 + 1, l, mid) + count_leq(ql, qr, x,
                node * 2 + 2, mid, r);
        }
        int count_greater(int l, int r, ll x){
            return r - l + 1 - count_leq(l, r, x);
        }
        ll sum_less(int l, int r, ll x){
            return sum_less(l, r + 1, x, 0, 0, n);
        }
        ll sum_less(int ql, int qr, ll x, int node, int l, int r){
            if (qr <= l || r <= ql) return 0;
            if (ql <= l && r <= qr) return sum_first(node, cnt_less(node, x));
            int mid = (l + r) >> 1;
            return sum_less(ql, qr, x, node * 2 + 1, l, mid) + sum_less(ql, qr, x,
                node * 2 + 2, mid, r);
        }
        ll sum_leq(int l, int r, ll x){
            return sum_leq(l, r + 1, x, 0, 0, n);
        }
        ll sum_leq(int ql, int qr, ll x, int node, int l, int r){
            if (qr <= l || r <= ql) return 0;
            if (ql <= l && r <= qr) return sum_first(node, cnt_leq(node, x));
            int mid = (l + r) >> 1;

```

```

        return sum_leq(ql, qr, x, node * 2 + 1, l, mid) + sum_leq(ql, qr, x,
            node * 2 + 2, mid, r);
    }
};

void SOLVE(){
}

```

Mo_s with updates.cpp

```

const int N = 2e5 + 5;
int n, q, block;
int a[N];
ll curAns, answer[N];

struct Query{
    int l, r, t, id;
    bool operator<(const Query &other) const{
        if (l / block != other.l / block) return l < other.l;
        if (r / block != other.r / block) return r < other.r;
        return t < other.t;
    }
};

struct Update{
    int i, old, val;
};

vector<Query> queries;
vector<Update> updates;
void add(int i){
}

void del(int i){
}

void apply(int t, int l, int r){
    auto &u = updates[t];
    if (l <= u.i && u.i <= r) del(u.i);
    a[u.i] = u.val;
    if (l <= u.i && u.i <= r) add(u.i);
}

void undo(int t, int l, int r){
    auto &u = updates[t];
    if (l <= u.i && u.i <= r) del(u.i);
    a[u.i] = u.old;
    if (l <= u.i && u.i <= r) add(u.i);
}

void process(){
    block = max(1, (int)pow(n, 2.0 / 3.0));
    sort(queries.begin(), queries.end());
    int l = 1, r = 0, t = 0;
    for (auto query : queries){
        while(t < query.t) apply(t++, l, r);
        while(t > query.t) undo(--t, l, r);
        while(r < query.r) add(++r);
        while(l > query.l) add(--l);
        while(r > query.r) del(r--);
        while(l < query.l) del(l++);
        answer[query.id] = curAns;
    }
}

void clear(){
    queries.clear();
    updates.clear();
    curAns = 0;
}

void SOLVE(){
    cin >> n >> q;
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<int> cur(a, a + n + 1);
    int id = 0;
    for (int i = 0; i < q; i++){
        char type;
        cin >> type;
        if (type == 'Q'){
            int l, r;
            cin >> l >> r;
            queries.push_back({l, r, (int)updates.size(), id++});
        }else{
            int idx, val;
            cin >> idx >> val;
            updates.push_back({idx, cur[idx], val});
            cur[idx] = val;
        }
    }
    process();
    for (int i = 0; i < id; i++){
        cout << answer[i] << endl;
    }
    clear();
}

```

Mo_s_algorithm.cpp

```

const int N = 1e5 + 5, sq = 317;
struct query{
    int l, r, i;
    bool operator<(const query &other) const {
        if (l / sq != other.l / sq)
            return l / sq < other.l / sq;
        return (l / sq & 1 ? r < other.r : other.r < r);
    }
};

void SOLVE() {
}

```

Multiset lazy deletion.cpp

```

template<typename T, typename Compare = less<T>>

```

```

struct MS {
    priority_queue<T, vector<T>, Compare> pq, del;
    void normalize(){
        while(!pq.empty() && !del.empty() && pq.top() == del.top()){
            pq.pop();
            del.pop();
        }
    }
    bool empty() { normalize(); return size() == 0; }
    int size() { return (int)pq.size() - (int)del.size(); }
    void insert(T x) { pq.push(x); }
    void erase(T x) { del.push(x); }
    T top() { normalize(); return pq.top(); }
    void pop() { normalize(); pq.pop(); }
    void clear() {
        while(!pq.empty()) pq.pop();
        while(!del.empty()) del.pop();
    }
};

void SOLVE() {
}

```

Offline 2D BIT.cpp

```

template<typename T>
struct OfflineBIT2D{
    int n;
    vector<int> head, vals;
    vector<T> bit;
    OfflineBIT2D(){
        OfflineBIT2D(int _n, vector<array<int, 2>> todo){
            init(_n, todo);
        }
        int ind(int r, int c){
            int start = head[r], end = head[r + 1];
            return upper_bound(vals.begin() + start, vals.end(), c) -
                vals.begin() - start - 1;
        }
        void init(int _n, vector<array<int, 2>> todo){
            n = _n;
            sort(todo.begin(), todo.end(), [](array<int, 2> &a, array<int, 2> &b){
                return a[1] < b[1];
            });
            vector<int> last(n + 1, -1), sz(n + 1, 1);
            for (auto [r, c] : todo){
                r++, c++;
                for (; r <= n; r += r & -r){
                    if (last[r] != c){
                        sz[r]++;
                        last[r] = c;
                    }
                }
            }
            head.assign(n + 2, 0);
            for (int i = 1; i <= n; i++){
                head[i + 1] = head[i] + sz[i];
            }
            vals.assign(head[n + 1], 0);
            bit.assign(head[n + 1], 0);
            last.assign(n + 1, -1);
            vector<int> cur = head;
            for (int i = 1; i <= n; i++){
                vals[cur[i]++] = 0;
            }
            for (auto [r, c] : todo){
                r++, c++;
                for (; r <= n; r += r & -r){
                    if (last[r] != c){
                        vals[cur[r]++] = c;
                        last[r] = c;
                    }
                }
            }
        }
        void add(int r, int c, T val){
            r++, c++;
            for (; r <= n; r += r & -r){
                int i = ind(r, c);
                int sz = head[r + 1] - head[r];
                for (; i < sz; i += i & -i){
                    bit[head[r] + i] += val;
                }
            }
        }
        T rect_sum(int r, int c){
            r++, c++;
            T ret = 0;
            for (; r > 0; r -= r & -r){
                int i = ind(r, c);
                for (; i > 0; i -= i & -i){
                    ret += bit[head[r] + i];
                }
            }
            return ret;
        }
        T rect_sum(int r1, int c1, int r2, int c2){
            if (r1 > r2 || c1 > c2) return 0;
            return rect_sum(r2, c2) - rect_sum(r2, c1 - 1) - rect_sum(r1 - 1, c2) +
                rect_sum(r1 - 1, c1 - 1);
        }
        T sum(int r, int c){
            return rect_sum(r, c);
        }
        T sum(int r1, int c1, int r2, int c2){
            return rect_sum(r1, c1, r2, c2);
        }
};

void SOLVE(){
}

```

```

}

```

ordered_set.cpp

```

template<class T>
using ordered_set = tree<T, null_type, std::less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

void SOLVE() {
}

```

Persistent DSU.cpp

```

// Add edges in nondecreasing time. Query with time 0 for the initial graph.
struct PersistentDSU{
    const int INF = 1e9;
    int n, timer;
    vector<int> parent, parent_time;
    vector<vector<pair<int, int>>> sz;
    vector<pair<int, int>> components_hist;
    PersistentDSU(){
        PersistentDSU(int _n){
            init(_n);
        }
        void init(int _n){
            n = _n;
            timer = 0;
            parent.resize(n + 1);
            parent_time.assign(n + 1, INF);
            sz.assign(n + 1, {});
            components_hist = {{0, n}};
            for (int i = 1; i <= n; i++){
                parent[i] = i;
                sz[i].push_back({0, 1});
            }
        }
        int find(int v, int t){
            return parent_time[v] > t ? v : find(parent[v], t);
        }
        int get_size(int v, int t){
            v = find(v, t);
            auto &h = sz[v];
            int p = upper_bound(h.begin(), h.end(), make_pair(t, INF)) - h.begin() -
                1;
            return h[p].second;
        }
        int components(int t){
            int p = upper_bound(components_hist.begin(), components_hist.end(),
                make_pair(t, INF)) - components_hist.begin() - 1;
            return components_hist[p].second;
        }
        bool same(int u, int v, int t){
            return find(u, t) == find(v, t);
        }
        bool unite(int a, int b, int t){
            timer = max(timer, t);
            a = find(a, t);
            b = find(b, t);
            if (a == b) return false;
            int sa = get_size(a, t), sb = get_size(b, t);
            if (sa < sb){
                swap(a, b);
                swap(sa, sb);
            }
            parent[b] = a;
            parent_time[b] = t;
            sz[a].push_back({t, sa + sb});
            components_hist.push_back({t, components(t) - 1});
            return true;
        }
        bool add_edge(int a, int b, int t){
            return unite(a, b, t);
        }
        bool unite(int a, int b){
            return unite(a, b, timer + 1);
        }
        bool add_edge(int a, int b){
            return unite(a, b);
        }
        int when_connected(int u, int v, int hi){
            if (!same(u, v, hi)) return -1;
            int l = 0, r = hi;
            while (l < r){
                int mid = (l + r) >> 1;
                if (same(u, v, mid)) r = mid;
                else l = mid + 1;
            }
            return l;
        }
        int when_connected(int u, int v){
            return when_connected(u, v, timer);
        }
};

void SOLVE(){
}

```

PST.cpp

```

struct PST{
    struct Node{ int l, r; ll val; };
    int n;
    vector<Node> tree;
    PST(){
        PST(int _n, int m = 0){
            init(_n, m);
        }
        void init(int _n, int m = 0){
            n = _n;

```

```

    tree.clear();
    if(m) tree.reserve(m);
    tree.push_back({0, 0, 0});
}
int clone(int node){
    tree.push_back(tree[node]);
    return (int)tree.size() - 1;
}
int update(int node, int idx, ll val){
    return update(node, idx, val, 0, n - 1);
}
int update(int node, int idx, ll val, int l, int r){
    int cur = clone(node);
    tree[cur].val += val;
    if (l == r) return cur;
    int mid = (l + r) >> 1;
    if (idx <= mid) tree[cur].l = update(tree[cur].l, idx, val, l, mid);
    else tree[cur].r = update(tree[cur].r, idx, val, mid + 1, r);
    return cur;
}
ll query(int node, int ql, int qr){
    return query(node, ql, qr, 0, n - 1);
}
ll query(int node, int ql, int qr, int l, int r){
    if (!node || qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return tree[node].val;
    int mid = (l + r) >> 1;
    return query(tree[node].l, ql, qr, l, mid) + query(tree[node].r, ql, qr,
        mid + 1, r);
}
int kth(int L, int R, ll k){
    return kth(L, R, k, 0, n - 1);
}
int kth(int L, int R, ll k, int l, int r){
    if (l == r) return l;
    int mid = (l + r) >> 1;
    ll left = tree[tree[R].l].val - tree[tree[L].l].val;
    if (k <= left) return kth(tree[L].l, tree[R].l, k, l, mid);
    return kth(tree[L].r, tree[R].r, k - left, mid + 1, r);
}
}
void SOLVE(){
}

```

seg_tree (lazy generic).cpp

```

struct node {
    ll lazy = 0, val = 0;
    bool isLazy = 0;
    node() {}
    node(ll x) : val(x) {}
    void add(ll x, int lx, int rx) {
        val += x; // for sum use: val += x * (rx - lx)
        lazy += x;
        isLazy = 1;
    }
};

#define lnode (x * 2 + 1)
#define rnode (x * 2 + 2)
#define md (lx + (rx - lx) / 2)
struct segment_tree {
    int n;
    vector<node> seg;
    segment_tree(int sz) {
        n = 1;
        while (n < sz) n *= 2;
        seg.assign(n * 2, node());
    }
    node merge(node &l, node &r) {
        node ret;
        ret.val = l.val + r.val; // min/max: change this
        return ret;
    }
    void build(vector<int> &v, int x, int lx, int rx) {
        if (rx - lx == 1) {
            if (lx < v.size()) seg[x] = node(v[lx]);
            return;
        }
        build(v, lnode, lx, md);
        build(v, rnode, md, rx);
        seg[x] = merge(seg[lnode], seg[rnode]);
    }
    void build(vector<int> &v) { build(v, 0, 0, n); }
    void propagate(int x, int lx, int rx) {
        if (rx - lx == 1 || !seg[x].isLazy) return;
        seg[lnode].add(seg[x].lazy, lx, md);
        seg[rnode].add(seg[x].lazy, md, rx);
        seg[x].isLazy = seg[x].lazy = 0;
    }
    void update(int l, int r, ll val, int x, int lx, int rx) {
        propagate(x, lx, rx);
        if (lx >= r || rx <= l) return;
        if (lx >= l && rx <= r) return seg[x].add(val, lx, rx);
        update(l, r, val, lnode, lx, md);
        update(l, r, val, rnode, md, rx);
        seg[x] = merge(seg[lnode], seg[rnode]);
    }
    void update(int l, int r, ll val) { update(l, r + 1, val, 0, 0, n); }
    node query(int l, int r, int x, int lx, int rx) {
        propagate(x, lx, rx);
        if (lx >= l && rx <= r) return seg[x];
        if (lx >= r || rx <= l) return node();
        node L = query(l, r, lnode, lx, md);
        node R = query(l, r, rnode, md, rx);
        return merge(L, R);
    }
    node query(int l, int r) {
        return query(l, r + 1, 0, 0, n);
    }
};

```

```

void SOLVE(){
}

```

seg_tree (lazy propagation).cpp

```

struct node {
    ll lazyMul = 1, lazyA = 0, lazyB = 0, val = 0;
    bool isLazy = 0;
    node() {}
    node(ll x) : val(x) {}
    ll sumIdx(ll lx, ll rx) {
        return (lx + rx - 1) * (rx - lx) / 2;
    }
    void apply(ll mul, ll a, ll b, int lx, int rx) {
        val = val * mul + a * sumIdx(lx, rx) + b * (rx - lx);
        lazyMul *= mul;
        lazyA = lazyA * mul + a;
        lazyB = lazyB * mul + b;
        isLazy = 1;
    }
    void add(ll x, int lx, int rx) { apply(1, 0, x, lx, rx); }
    void multiply(ll x, int lx, int rx) { apply(x, 0, 0, lx, rx); }
    void set(ll x, int lx, int rx) { apply(0, 0, x, lx, rx); }
    void add_ap(ll first, ll diff, int l, int lx, int rx) {
        apply(1, diff, first - diff * l, lx, rx);
    }
};

#define lnode (x * 2 + 1)
#define rnode (x * 2 + 2)
#define md (lx + (rx - lx) / 2)
struct segment_tree {
    int n;
    vector<node> seg;
    segment_tree(int sz) {
        n = 1;
        while (n < sz) n *= 2;
        seg.assign(n * 2, node());
    }
    node merge(node &l, node &r) {
        node ret;
        ret.val = l.val + r.val;
        return ret;
    }
    void build(vector<int> &v, int x, int lx, int rx) {
        if (rx - lx == 1) {
            if (lx < v.size()) seg[x] = node(v[lx]);
            return;
        }
        build(v, lnode, lx, md);
        build(v, rnode, md, rx);
        seg[x] = merge(seg[lnode], seg[rnode]);
    }
    void build(vector<int> &v) { build(v, 0, 0, n); }
    void propagate(int x, int lx, int rx) {
        if (rx - lx == 1 || !seg[x].isLazy) return;
        seg[lnode].apply(seg[x].lazyMul, seg[x].lazyA, seg[x].lazyB, lx, md);
        seg[rnode].apply(seg[x].lazyMul, seg[x].lazyA, seg[x].lazyB, md, rx);
        seg[x].lazyMul = 1;
        seg[x].lazyA = seg[x].lazyB = 0;
        seg[x].isLazy = 0;
    }
    void update(int l, int r, ll mul, ll a, ll b, int x, int lx, int rx) {
        propagate(x, lx, rx);
        if (lx >= r || rx <= l) return;
        if (lx >= l && rx <= r) return seg[x].apply(mul, a, b, lx, rx);
        update(l, r, mul, a, b, lnode, lx, md);
        update(l, r, mul, a, b, rnode, md, rx);
        seg[x] = merge(seg[lnode], seg[rnode]);
    }
    void update(int l, int r, ll val) { add(l, r, val); }
    void add(int l, int r, ll val) { update(l, r + 1, 1, 0, val, 0, 0, n); }
    void multiply(int l, int r, ll val) { update(l, r + 1, val, 0, 0, 0, n); }
    void set(int l, int r, ll val) { update(l, r + 1, 0, 0, val, 0, 0, n); }
    void add_ap(int l, int r, ll first, ll diff) {
        update(l, r + 1, 1, diff, first - diff * l, 0, 0, n);
    }
    node query(int l, int r, int x, int lx, int rx) {
        propagate(x, lx, rx);
        if (lx >= l && rx <= r) return seg[x];
        if (lx >= r || rx <= l) return node();
        node L = query(l, r, lnode, lx, md);
        node R = query(l, r, rnode, md, rx);
        return merge(L, R);
    }
    node query(int l, int r) {
        return query(l, r + 1, 0, 0, n);
    }
};

void SOLVE() {
}

```

seg_tree (point, range query).cpp

```

struct node{
    ll val;
    node(){}
    node(ll v): val(v) {}
};

struct segment_tree{
    int n;
    vector<node> tree;
    node NEUTRAL;
    function<node(const node&, const node&>> merge;
    segment_tree(int n_, node neutral,
        function<node(const node&, const node&>> merge_)
        : NEUTRAL(neutral), merge(merge_){

```

```

n = 1;
while (n < n_) n <= 1;
tree.assign(n << 1, NEUTRAL);
}
void set(int i, ll val, int pos, int l, int r){
    if (r - l == 1) { tree[pos] = node(val); return; }
    int mid = (l + r) >> 1;
    if (i < mid) set(i, val, 2 * pos + 1, l, mid);
    else set(i, val, 2 * pos + 2, mid, r);
    tree[pos] = merge(tree[2 * pos + 1], tree[2 * pos + 2]);
}
void set(int i, ll val){ set(i, val, 0, 0, n); }
node query(int l, int r, int pos, int nl, int nr){
    if (nr <= l || r <= nl) return NEUTRAL;
    if (l <= nl && nr <= r) return tree[pos];
    int mid = (nl + nr) >> 1;
    return merge(query(l, r, 2 * pos + 1, nl, mid), query(l, r, 2 * pos + 2,
        ↪ mid, nr));
}
node query(int l, int r){ return query(l, r + 1, 0, 0, n); }
};

void SOLVE() {
}

```

sparse_table.cpp

```

template<class T>
struct SparseTable{
    int n, LOG;
    vector<int> lg;
    vector<vector<T>>> table;
    function<T(T, T)> f;
    SparseTable(){}
    SparseTable(vector<T> &v, function<T(T, T)> func){
        init(v, func);
    }
    void init(vector<T> &v, function<T(T, T)> func){
        n = v.size();
        f = func;
        LOG = __lg(n) + 1;
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++){
            lg[i] = lg[i >> 1] + 1;
        }
        table.assign(LOG, vector<T>(n));
        table[0] = v;
        for (int j = 1; j < LOG; j++){
            for (int i = 0; i + (1 << j) <= n; i++){
                table[j][i] = f(table[j - 1][i], table[j - 1][i + (1 << j -
                    ↪ 1)]));
            }
        }
    }
    T query(int l, int r){
        if (l > r) swap(l, r);
        int j = lg[r - l + 1];
        return f(table[j][l], table[j][r - (1 << j) + 1]);
    }
};

void SOLVE(){
}

```

sqrt_decomposition.cpp

```

struct Sq_dec{
    int bs;
    vector<ll> a, B;
    Sq_dec(vector<ll> &arr){
        int n = arr.size(); B.assign(n, 0); a = arr; bs = ceil(sqrt(arr.size()));
        for (int i = 0; i < a.size(); i++){
            B[i / bs] += a[i];
        }
    }
    void update(int x, int v){
        B[x / bs] -= a[x]; a[x] = v; B[x / bs] += a[x];
    }
    ll query(int r){
        ll res = 0;
        for (int i = 0; i < r / bs; i++){ res += B[i]; }
        for (int i = (r / bs) * bs; i < r; i++){ res += a[i]; }
        return res;
    }
    ll query(int l, int r) { return query(r) - query(l - 1); }
};

void SOLVE() {
}

```

topK.cpp

```

struct TopKSum{
    int K;
    multiset<ll> top_k, rest;
    ll sum;
    TopKSum(){}
    TopKSum(int k){
        init(k);
    }
    void init(int k){
        K = k;
        sum = 0;
        top_k.clear();
        rest.clear();
    }
    void fix(){
        while ((int)top_k.size() > K){

```

```

            auto it = top_k.begin();
            sum -= *it;
            rest.insert(*it);
            top_k.erase(it);
        }
        while ((int)top_k.size() < K && !rest.empty()){
            auto it = prev(rest.end());
            sum += *it;
            top_k.insert(*it);
            rest.erase(it);
        }
        while (!top_k.empty() && !rest.empty() && *top_k.begin() <
            ↪ *prev(rest.end())){
            auto small = top_k.begin();
            auto big = prev(rest.end());
            ll x = *small, y = *big;
            sum += y - x;
            top_k.erase(small);
            rest.erase(big);
            top_k.insert(y);
            rest.insert(x);
        }
    }
    void setK(int k){
        K = k;
        fix();
    }
    void insert(ll val){
        if (K == 0){
            rest.insert(val);
            return;
        }
        top_k.insert(val);
        sum += val;
        fix();
    }
    void erase(ll val){
        auto it = rest.find(val);
        if (it != rest.end()){
            rest.erase(it);
        }else{
            it = top_k.find(val);
            if (it != top_k.end()){
                sum -= val;
                top_k.erase(it);
            }
        }
        fix();
    }
    int size(){
        return (int)top_k.size() + (int)rest.size();
    }
    void clear(){
        top_k.clear();
        rest.clear();
        sum = 0;
    }
    ll get(){
        return sum;
    }
};

void SOLVE(){
}

```

TwoStackQ.cpp

```

const ll inf = 1e18;
struct TwoStackQ {
    struct Node {
        ll mx = -inf, mn = inf, val;
        Node(): val(0) { }
        Node(ll x): mx(x), mn(x), val(x) { }
    };
    stack<Node> a, b;
    int size() { return a.size() + b.size(); }
    bool empty() { return size() == 0; }
    void clear() {
        while (!a.empty()) a.pop();
        while (!b.empty()) b.pop();
    }
    void mrg(Node& a, Node& b) {
        a.mn = min(a.mn, b.mn);
        a.mx = max(a.mx, b.mx);
    }
    void push(ll val) {
        auto nd = Node(val);
        if (!a.empty()) mrg(nd, a.top());
        a.push(nd);
    }
    void move() {
        while (!a.empty()) {
            auto nd = Node(a.top().val);
            if (!b.empty()) mrg(nd, b.top());
            b.push(nd), a.pop();
        }
    }
    Node get() {
        Node res;
        if (!b.empty()) mrg(res, b.top());
        if (!a.empty()) mrg(res, a.top());
        return res;
    }
    void pop() {
        if (b.empty()) move();
        if (!b.empty()) b.pop();
    }
};

void SOLVE() {
}

```

}

3 graphs

2SAT.cpp

```
struct two_sat{
    int n;
    vector<vector<int>> g, gr;
    vector<int> comp, top, answer;
    vector<bool> vis;
    two_sat(){}
    two_sat(int _n){
        init(_n);
    }
    void init(int _n){
        n = _n;
        g.assign(2 * n, {});
        gr.assign(2 * n, {});
        comp.assign(2 * n, 0);
        vis.assign(2 * n, false);
        answer.assign(n, 0);
        top.clear();
    }
    int var(int i, bool val){
        return i + (val ? 0 : n);
    }
    void add_edge(int u, int v){
        g[u].push_back(v);
        gr[v].push_back(u);
    }
    void add_imply(int i, bool f, int j, bool k){
        add_edge(var(i, f), var(j, k));
    }
    void set_true(int i){
        add_imply(i, false, i, true);
    }
    void set_false(int i){
        add_imply(i, true, i, false);
    }
    void add_or(int i, bool f, int j, bool k){
        add_imply(i, !f, j, k);
        add_imply(j, !k, i, f);
    }
    void add_xor(int i, bool f, int j, bool k){
        add_or(i, f, j, k);
        add_or(i, !f, j, !k);
    }
    void add_xnor(int i, bool f, int j, bool k){
        add_or(i, f, j, !k);
        add_or(i, !f, j, k);
    }
    void add_and(int i, bool f, int j, bool k){
        add_imply(i, !f, i, f);
        add_imply(j, !k, j, k);
    }
    void dfs(int u){
        vis[u] = true;
        for (auto v : g[u]){
            if (!vis[v]) dfs(v);
        }
        top.push_back(u);
    }
    void scc(int u, int id){
        vis[u] = true;
        comp[u] = id;
        for (auto v : gr[u]){
            if (!vis[v]) scc(v, id);
        }
    }
    bool satisfiable(){
        top.clear();
        fill(vis.begin(), vis.end(), false);
        for (int i = 0; i < 2 * n; i++){
            if (!vis[i]) dfs(i);
        }
        fill(vis.begin(), vis.end(), false);
        reverse(top.begin(), top.end());
        int id = 0;
        for (auto v : top){
            if (!vis[v]) scc(v, id++);
        }
        for (int i = 0; i < n; i++){
            if (comp[i] == comp[i + n]) return false;
            answer[i] = comp[i] > comp[i + n];
        }
        return true;
    }
};

void SOLVE(){
}
```

Articulation points.cpp

```
struct ArticulationPoints{
    int n, timer;
    vector<vector<pair<int, int>>> graph;
    vector<pair<int, int>>> edges;
    vector<int> tin, low, isCut;
    ArticulationPoints(){}
    ArticulationPoints(int _n){
        init(_n);
    }
    void init(int _n){
        n = _n;
        timer = 0;
        graph.assign(n + 1, {});
        edges.clear();
        tin.assign(n + 1, 0);
        low.assign(n + 1, 0);
        isCut.assign(n + 1, 0);
    }
    void add_edge(int u, int v){
```

```
        int id = edges.size();
        edges.push_back({u, v});
        graph[u].push_back({v, id});
        graph[v].push_back({u, id});
    }
    void dfs(int v, int pe = -1){
        tin[v] = low[v] = ++timer;
        int children = 0;
        for (auto [u, id] : graph[v]){
            if (id == pe) continue;
            if (tin[u]){
                low[v] = min(low[v], tin[u]);
            }else{
                dfs(u, id);
                low[v] = min(low[v], low[u]);
                if (pe != -1 && low[u] >= tin[v]){
                    isCut[v] = true;
                }
                children++;
            }
        }
        if (pe == -1 && children > 1){
            isCut[v] = true;
        }
    }
    vector<int> build(){
        timer = 0;
        fill(tin.begin(), tin.end(), 0);
        fill(low.begin(), low.end(), 0);
        fill(isCut.begin(), isCut.end(), 0);
        for (int i = 1; i <= n; i++){
            if (!tin[i]) dfs(i);
        }
        vector<int> ret;
        for (int i = 1; i <= n; i++){
            if (isCut[i]) ret.push_back(i);
        }
        return ret;
    }
};

void SOLVE(){
}
```

Biconnected components.cpp

```
struct BiconnectedComponents{
    int n, timer;
    vector<vector<pair<int, int>>> graph;
    vector<pair<int, int>>> edges;
    vector<vector<int>> components, comp_edges;
    vector<int> tin, low, isCut, st;
    BiconnectedComponents(){}
    BiconnectedComponents(int _n){
        init(_n);
    }
    void init(int _n){
        n = _n;
        timer = 0;
        graph.assign(n + 1, {});
        edges.clear();
        components.clear();
        comp_edges.clear();
        tin.assign(n + 1, 0);
        low.assign(n + 1, 0);
        isCut.assign(n + 1, 0);
        st.clear();
    }
    void add_edge(int u, int v){
        int id = edges.size();
        edges.push_back({u, v});
        graph[u].push_back({v, id});
        graph[v].push_back({u, id});
    }
    void add_component(int until){
        vector<int> verts, eids;
        while (true){
            int id = st.back();
            st.pop_back();
            eids.push_back(id);
            verts.push_back(edges[id].first);
            verts.push_back(edges[id].second);
            if (id == until) break;
        }
        sort(verts.begin(), verts.end());
        verts.erase(unique(verts.begin(), verts.end()), verts.end());
        components.push_back(verts);
        comp_edges.push_back(eids);
    }
    void dfs(int v, int pe = -1){
        tin[v] = low[v] = ++timer;
        int children = 0;
        for (auto [u, id] : graph[v]){
            if (id == pe) continue;
            if (!tin[u]){
                st.push_back(id);
                dfs(u, id);
                low[v] = min(low[v], low[u]);
                if (low[u] >= tin[v]){
                    if (pe != -1 || children > 0) isCut[v] = true;
                    add_component(id);
                }
                children++;
            }else if (tin[u] < tin[v]){
                st.push_back(id);
                low[v] = min(low[v], tin[u]);
            }
        }
        if (pe == -1 && children > 1){
            isCut[v] = true;
        }
    }
};
```

```

void build(){
    timer = 0;
    components.clear();
    comp_edges.clear();
    st.clear();
    fill(tin.begin(), tin.end(), 0);
    fill(low.begin(), low.end(), 0);
    fill(isCut.begin(), isCut.end(), 0);
    for (int i = 1; i <= n; i++){
        if (tin[i]) continue;
        if (graph[i].empty()){
            components.push_back({i});
            comp_edges.push_back({});
        }else{
            dfs(i);
        }
    }
}

void SOLVE(){
}

```

Bipartite check.cpp

```

struct Bipartite{
    int n;
    bool ok;
    vector<vector<int>> graph;
    vector<int> color;
    Bipartite(){}
    Bipartite(int _n){
        init(_n);
    }
    void init(int _n){
        n = _n;
        ok = true;
        graph.assign(n + 1, {});
        color.assign(n + 1, -1);
    }
    void add_edge(int u, int v){
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    bool build(){
        fill(color.begin(), color.end(), -1);
        ok = true;
        queue<int> q;
        for (int i = 1; i <= n; i++){
            if (color[i] != -1) continue;
            color[i] = 0;
            q.push(i);
            while (!q.empty()){
                int v = q.front();
                q.pop();
                for (auto u : graph[v]){
                    if (color[u] == -1){
                        color[u] = color[v] ^ 1;
                        q.push(u);
                    }else if (color[u] == color[v]){
                        ok = false;
                    }
                }
            }
        }
        return ok;
    }
    vector<int> get_side(int c){
        vector<int> ret;
        for (int i = 1; i <= n; i++){
            if (color[i] == c) ret.push_back(i);
        }
        return ret;
    }
};

void SOLVE(){
}

```

Blossom.cpp

```

struct blossom {
    int n, m;
    vector<int> mate;
    vector<vector<int>> b;
    vector<int> p, d, bl;
    vector<vector<int>> g;
    blossom(int n) : n(n) {
        m = n + n / 2;
        mate.assign(m, -1);
        b.resize(m);
        p.resize(m);
        d.resize(m);
        bl.resize(m);
        g.assign(m, vector<int>(m, -1));
    }
    void add_edge(int u, int v) {
        g[u][v] = u;
        g[v][u] = v;
    }
    void match(int u, int v) {
        g[u][v] = g[v][u] = -1;
        mate[u] = v;
        mate[v] = u;
    }
    vector<int> trace(int x) {
        vector<int> vx;
        while(true) {
            while(bl[x] != x) x = bl[x];
            if(!vx.empty() && vx.back() == x) break;

```

```

            vx.push_back(x);
            x = p[x];
        }
        return vx;
    }
    void contract(int c, int x, int y, vector<int> &vx, vector<int> &vy) {
        b[c].clear();
        int r = vx.back();
        while(!vx.empty() && !vy.empty() && vx.back() == vy.back()) {
            r = vx.back();
            vx.pop_back();
            vy.pop_back();
        }
        b[c].push_back(r);
        b[c].insert(b[c].end(), vx.rbegin(), vx.rend());
        b[c].insert(b[c].end(), vy.begin(), vy.end());
        for(int i = 0; i <= c; i++) {
            g[c][i] = g[i][c] = -1;
        }
        for(int z : b[c]) {
            bl[z] = c;
            for(int i = 0; i < c; i++) {
                if(g[z][i] != -1) {
                    g[c][i] = z;
                    g[i][c] = g[i][z];
                }
            }
        }
    }
    vector<int> lift(vector<int> &vx) {
        vector<int> A;
        while(vx.size() >= 2) {
            int z = vx.back(); vx.pop_back();
            if(z < n) {
                A.push_back(z);
                continue;
            }
            int w = vx.back();
            int i = (A.size() % 2 == 0 ? find(b[z].begin(), b[z].end(), g[z][w])
                : -b[z].begin() : 0);
            int j = (A.size() % 2 == 1 ? find(b[z].begin(), b[z].end(),
                : g[z][A.back()]) - b[z].begin() : 0);
            int k = b[z].size();
            int dif = (A.size() % 2 == 0 ? i % 2 == 1 : j % 2 == 0) ? 1 : k - 1;
            while(i != j) {
                vx.push_back(b[z][i]);
                i = (i + dif) % k;
            }
            vx.push_back(b[z][i]);
        }
        return A;
    }
    int solve() {
        for(int ans = 0; ; ans++) {
            fill(d.begin(), d.end(), 0);
            queue<int> Q;
            for(int i = 0; i < m; i++) bl[i] = i;
            for(int i = 0; i < n; i++) {
                if(mate[i] == -1) {
                    Q.push(i);
                    p[i] = i;
                    d[i] = 1;
                }
            }
            int c = n;
            bool aug = false;
            while(!Q.empty() && !aug) {
                int x = Q.front(); Q.pop();
                if(bl[x] != x) continue;
                for(int y = 0; y < c; y++) {
                    if(bl[y] == y && g[x][y] != -1) {
                        if(d[y] == 0) {
                            p[y] = x;
                            d[y] = 2;
                            p[mate[y]] = y;
                            d[mate[y]] = 1;
                            Q.push(mate[y]);
                        }else if(d[y] == 1) {
                            vector<int> vx = trace(x);
                            vector<int> vy = trace(y);
                            if(vx.back() == vy.back()) {
                                contract(c, x, y, vx, vy);
                                Q.push(c);
                                p[c] = p[b[c][0]];
                                d[c] = 1;
                                c++;
                            }else {
                                aug = true;
                                vx.insert(vx.begin(), y);
                                vy.insert(vy.begin(), x);
                                vector<int> A = lift(vx);
                                vector<int> B = lift(vy);
                                A.insert(A.end(), B.rbegin(), B.rend());
                                for(int i = 0; i < (int) A.size(); i += 2) {
                                    match(A[i], A[i + 1]);
                                    if(i + 2 < (int) A.size()) add_edge(A[i + 1]
                                        : 1], A[i + 2]);
                                }
                            }
                            break;
                        }
                    }
                }
            }
            if(!aug) return ans;
        }
    }
};

void SOLVE(){
}

```

Bridges.cpp

```
const int N = 2e5 + 5;
vector<pair<int, int>> graph[N];
vector<pair<int, int>> edges, bridges;
vector<bool> isBridge;
int tin[N], low[N], timer;

void add_edge(int u, int v){
    int id = edges.size();
    edges.push_back({u, v});
    graph[u].push_back({v, id});
    graph[v].push_back({u, id});
}

void dfs(int v, int pe = -1){
    tin[v] = low[v] = ++timer;
    for (auto [u, id] : graph[v]){
        if (id == pe) continue;
        if (tin[u]){
            low[v] = min(low[v], tin[u]);
        } else {
            dfs(u, id);
            low[v] = min(low[v], low[u]);
            if (low[u] > tin[v]){
                isBridge[id] = true;
                bridges.push_back(edges[id]);
            }
        }
    }
}

void build(int n){
    timer = 0;
    bridges.clear();
    isBridge.assign(edges.size(), false);
    for (int i = 1; i <= n; i++){
        tin[i] = low[i] = 0;
    }
    for (int i = 1; i <= n; i++){
        if (!tin[i]) dfs(i);
    }
}

void clear(int n){
    for (int i = 1; i <= n; i++){
        graph[i].clear();
        tin[i] = low[i] = 0;
    }
    edges.clear();
    bridges.clear();
    isBridge.clear();
    timer = 0;
}

void SOLVE(){
}
```

```
return st.order_of_key({x, INT_MAX});
}

ll countWith(int v){
    ll ret = 0;
    for (int i = 0; i < (int)up[v].size(); i++){
        auto [c, d] = up[v][i];
        auto [id, _] = side[v][i];
        ret += get(bucket[c], k - d);
        if (id) ret -= get(sideBucket[id], k - d);
    }
    return ret;
}

void add(int v){
    if (active[v]) return;
    paths += countWith(v);
    active[v] = true;
    for (int i = 0; i < (int)up[v].size(); i++){
        auto [c, d] = up[v][i];
        auto [id, _] = side[v][i];
        bucket[c].insert({d, v});
        if (id) sideBucket[id].insert({d, v});
    }
}

void erase(int v){
    if (!active[v]) return;
    active[v] = false;
    for (int i = 0; i < (int)up[v].size(); i++){
        auto [c, d] = up[v][i];
        auto [id, _] = side[v][i];
        bucket[c].erase({d, v});
        if (id) sideBucket[id].erase({d, v});
    }
    paths -= countWith(v);
}

void toggle(int v){
    active[v] ? erase(v) : add(v);
}

void clear(int n){
    for (int i = 1; i <= n; i++){
        graph[i].clear();
        up[i].clear();
        side[i].clear();
        bucket[i].clear();
        rem[i] = active[i] = false;
        sz[i] = par[i] = 0;
    }
    sideBucket.assign(1, ordered_set<pair<ll, int>>());
    sideId = 0;
    paths = 0;
}

void SOLVE(){
}
```

Centroid decomposition buckets.cpp

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template<class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

const int N = 1e5 + 5;
vector<pair<int, ll>> graph[N];
vector<pair<int, ll>> up[N], side[N]; // {centroid / side id, distance}
ordered_set<pair<ll, int>> bucket[N];
vector<ordered_set<pair<ll, int>>> sideBucket(1);
bool rem[N], active[N];
int sz[N], par[N], sideId;
ll k, paths;

int pre(int v, int p = -1){
    sz[v] = 1;
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u]) sz[v] += pre(u, v);
    }
    return sz[v];
}

int cen(int v, int tree_size, int p = -1){
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u] && sz[u] * 2 > tree_size){
            return cen(u, tree_size, v);
        }
    }
    return v;
}

void dfs(int v, int p, int c, ll d, int id){
    up[v].push_back({c, d});
    side[v].push_back({id, d});
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u]) dfs(u, v, c, d + w, id);
    }
}

void build(int v = 1, int p = 0){
    int c = cen(v, pre(v));
    par[c] = p;
    rem[c] = true;
    up[c].push_back({c, 0});
    side[c].push_back({0, 0});
    for (auto [u, w] : graph[c]){
        if (rem[u]) continue;
        sideBucket.emplace_back();
        dfs(u, c, c, w, ++sideId);
    }
    for (auto [u, w] : graph[c]){
        if (!rem[u]) build(u, c);
    }
}

int get(ordered_set<pair<ll, int>> &st, ll x){
    if (x < 0) return 0;
```

Centroid decomposition with updates.cpp

```
const int N = 1e5 + 5;
const ll INF = 4e18;
vector<pair<int, ll>> graph[N], path[N];
multiset<ll> st[N];
bool rem[N], active[N];
int sz[N], par[N];

int pre(int v, int p = -1){
    sz[v] = 1;
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u]) sz[v] += pre(u, v);
    }
    return sz[v];
}

int cen(int v, int tree_size, int p = -1){
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u] && sz[u] * 2 > tree_size){
            return cen(u, tree_size, v);
        }
    }
    return v;
}

void dfs(int v, int p, int c, ll d){
    path[v].push_back({c, d});
    for (auto [u, w] : graph[v]){
        if (u != p && !rem[u]) dfs(u, v, c, d + w);
    }
}

void build(int v = 1, int p = 0){
    int c = cen(v, pre(v));
    par[c] = p;
    rem[c] = true;
    dfs(c, 0, c, 0);
    for (auto [u, w] : graph[c]){
        if (!rem[u]) build(u, c);
    }
}

void add(int v){
    if (active[v]) return;
    active[v] = true;
    for (auto [c, d] : path[v]) st[c].insert(d);
}

void erase(int v){
    if (!active[v]) return;
    active[v] = false;
    for (auto [c, d] : path[v]){
        auto it = st[c].find(d);
        if (it != st[c].end()) st[c].erase(it);
    }
}

void toggle(int v){
    active[v] ? erase(v) : add(v);
}

ll query(int v){
    ll ans = INF;
    for (auto [c, d] : path[v]){
```

```

        if(!st[c].empty()) ans = min(ans, d + *st[c].begin());
    }
    return ans;
}

void clear(int n){
    for(int i = 1; i <= n; i++){
        graph[i].clear();
        path[i].clear();
        st[i].clear();
        rem[i] = active[i] = false;
        sz[i] = par[i] = 0;
    }
}

void SOLVE(){
}

```

Centroid decomposition.cpp

```

const int N = 2e5 + 5;
vector<int> graph[N];
bool rem[N];
int sz[N];
int answer;

int pre(int v, int p = -1){
    sz[v] = 1;
    for(auto &u : graph[v]){
        if(u == p || rem[u]) continue;
        sz[v] += pre(u, v);
    }
    return sz[v];
}

int cen(int v, int tree_size, int p = -1){
    for(auto &u : graph[v]){
        if(u == p || rem[u]) continue;
        if(sz[u] * 2 > tree_size) return cen(u, tree_size, v);
    }
    return v;
}

void process_centroid(int c);
void build(int v = 1){
    int centroid = cen(v, pre(v));
    process_centroid(centroid);
    rem[centroid] = true;
    for(auto &u : graph[centroid]){
        if(rem[u]) continue;
        build(u);
    }
}

void dfs_calc(int &v, int &p) {
    for(int &u : graph[v]){
        if(u == p || rem[u]) continue;
        dfs_calc(u, v);
    }
}

void dfs_add(int &v, int &p){
    for(int &u : graph[v]){
        if(u == p || rem[u]) continue;
        dfs_add(u, v);
    }
}

void process_centroid(int c){
    for(int &u : graph[c]) {
        if(rem[u]) continue;
        dfs_calc(u, c);
        dfs_add(u, c);
    }
    for(int &u : graph[c]){
        if(rem[u]) continue;
        dfs_add(u, c);
    }
}

void clear(int n){
    for(int i = 1; i <= n; i++){
        rem[i] = sz[i] = 0;
    }
}

void SOLVE() {
}

```

dijkstra.cpp

```

const ll INF = 4e18;
int n;
vector<vector<pair<int, ll>>> graph;

vector<ll> dijkstra(int src, vector<int> &parent){
    vector<ll> dist(n + 1, INF);
    parent.assign(n + 1, -1);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
    pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()){
        auto [d, v] = pq.top();
        pq.pop();
        if (d != dist[v]) continue;
        for (auto [u, w] : graph[v]){
            if (dist[u] > d + w){
                dist[u] = d + w;
                parent[u] = v;
                pq.push({dist[u], u});
            }
        }
    }
    return dist;
}

vector<ll> dijkstra(int src){
    vector<int> parent;
}

```

```

    return dijkstra(src, parent);
}

void SOLVE(){
}

```

Dinic.cpp

```

const ll inf = 1e18;
struct Edge {
    int u, v;
    ll cap, flow = 0;
    Edge(int u, int v, ll cap) : u(u), v(v), cap(cap) {}
};

struct Dinic {
    vector<Edge> E;
    vector<vector<int>>> adj;
    vector<int> level, ptr;
    queue<int> q;
    int n, m = 0, s, t;
    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int u, int v, ll cap) {
        E.push_back(Edge(u, v, cap));
        E.push_back(Edge(v, u, 0));
        adj[u].push_back(m);
        adj[v].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while (q.size()) {
            auto u = q.front();
            q.pop();
            for (auto &id : adj[u]) {
                if (E[id].cap == E[id].flow) continue;
                if (level[E[id].v] != -1) continue;
                level[E[id].v] = level[u] + 1;
                q.push(E[id].v);
            }
        }
        return level[t] != -1;
    }

    ll dfs(int u, ll pushed) {
        if (pushed == 0) return 0;
        if (u == t) return pushed;
        for (int &cid = ptr[u]; cid < (int)adj[u].size(); ++cid) {
            int id = adj[u][cid];
            ll v = E[id].v;
            if (level[u] + 1 != level[v]) continue;
            ll tr = dfs(v, min(pushed, E[id].cap - E[id].flow));
            if (tr == 0) continue;
            E[id].flow += tr;
            E[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }

    ll flow() {
        ll ret = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs()) break;
            fill(ptr.begin(), ptr.end(), 0);
            while (ll pushed = dfs(s, inf)) {
                ret += pushed;
            }
        }
        return ret;
    }

    // get any set of E to achieve the min cut (max flow)
    vector<pair<int, int>> getEdges() {
        vector<int> srcSide(n, 0);
        queue<int> q;
        q.push(s);
        srcSide[s] = 1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto &id : adj[u]) {
                if (!srcSide[E[id].v] && E[id].flow < E[id].cap) {
                    srcSide[E[id].v] = 1;
                    q.push(E[id].v);
                }
            }
        }
        vector<pair<int, int>> res;
        for (int u = 0; u < n; ++u) {
            if (!srcSide[u]) continue;
            for (auto &id : adj[u]) {
                if (!srcSide[E[id].v] && E[id].cap > 0)
                    res.push_back({u, E[id].v});
            }
        }
        return res;
    }

    // Get The Edges Used In Max Bipartite Matching : s(0), t(n + m + 1)
    vector<pair<int, int>> getMatchingEdges(int n, int m) {
        vector<pair<int, int>> ret;
        for (auto &e : E) {
            if (1 <= e.u && e.u <= n && n + 1 <= e.v && e.v <= n + m) {
                if (e.flow > 0) {
                    ret.push_back({e.u, e.v - n});
                }
            }
        }
        return ret;
    }
}

```

```
};

void SOLVE(){
}
```

DSU on trees.cpp

```
void SOLVE(){
    int n; cin >> n;
    vector<vector<int>> graph(n + 1);
    for(int i = 0, u, v; i < n - 1; i++){
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    vector<int> tin(n + 1), tout(n + 1), vertex(n + 1), sz(n + 1, 1), d(n + 1, 1);
    int timer = 0;
    function<int(int, int)> euler = [&](int u, int p){
        tin[u] = ++timer;
        vertex[tin[u]] = u;
        for(auto v : graph[u]) if(v != p){
            d[v] = d[u] + 1;
            sz[u] += euler(v, u);
        }
        tout[u] = timer;
        return sz[u];
    };
    euler(1, -1);

    auto add = [&](int u){};
    auto remove = [&](int u){};

    function<void(int, int, bool)> dfs = [&](int u, int p, bool keep){
        int big = -1;
        for(auto v : graph[u]){
            if(v == p) continue;
            if(big == -1 || sz[v] > sz[big]) big = v;
        }
        for(auto v : graph[u])
            if(v != p && v != big) dfs(v, u, 0);
        if(big != -1) dfs(big, u, 1);
        for(auto v : graph[u]){
            if(v == p || v == big) continue;
            for(int i = tin[v]; i <= tout[v]; i++){
                // here
                add(vertex[i]);
            }
        }
        add(u);
        if(!keep){
            for(int i = tin[u]; i <= tout[u]; i++) remove(vertex[i]);
        }
    };
    dfs(1, -1, 1);
}
```

eulerWalk.cpp

```
/*
undirected/cycle: every vertex is even
undirected/path: 0/2 vertices are odd.
directed/cycle: every vertex in == out
directed/path: exactly one (out - in == 1) -> src
                exactly one (in - out == 1) -> sink
                all other in == out.
*/
vector<int> eulerWalk(vector<vector<pair<int, int>>> &gr, int nedges, int src = 0){
    int n = gr.size();
    vector<int> D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = gr[x].size();
        if (it == end) {
            ret.push_back(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D) if (x < 0 || ret.size() != nedges + 1) return {};
    return {ret.rbegin(), ret.rend()};
}

void SOLVE(){
}
```

HLD.cpp

```
struct HLD {
    int root = 0;
    vector<vector<int>> g;
    explicit HLD(int n) : g(n) {}
    void add(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> &operator[](int u) {
        return g[u];
    }
    int timer = 0;
    vector<int> tin, tout, d, sz, head, par, vertex;
    void init(int rt = 0) {
```

```
        root = rt;
        tin = tout = d = head = par = vertex = vector<int>(g.size() + 1);
        sz.resize(g.size(), 1);
        par[root] = head[root] = root;
        dfs(root); dfs2(root);
    }
    void dfs(int u){
        for(int & v : g[u]){
            d[v] = d[u] + 1;
            par[v] = u;
            g[v].erase(find(g[v].begin(), g[v].end(), u));
            dfs(v);
            sz[u] += sz[v];
            if(sz[v] > sz[g[u][0]]) swap(v, g[u][0]);
        }
    }
    void dfs2(int u){
        tin[u] = ++timer;
        vertex[tin[u]] = u;
        for(int v : g[u]){
            head[v] = v == g[u][0] ? head[u] : v;
            dfs2(v);
        }
        tout[u] = timer;
    }
    int jump(int u, int k) {
        if(k > d[u]) return -1;
        int depth = d[u] - k;
        while (d[head[u]] > depth) { u = par[head[u]]; }
        return vertex[tin[u] - d[u] + depth];
    }
    bool isAnc(int u, int v) {
        return tin[u] <= tin[v] && tin[v] <= tout[u];
    }
    int lca(int u, int v) {
        if(d[u] > d[v]) swap(u, v);
        if(isAnc(u, v)) return u;
        while (head[u] != head[v]){
            if (d[head[u]] > d[head[v]]) u = par[head[u]];
            else v = par[head[v]];
        }
        return d[u] < d[v] ? u : v;
    }
    int dis(int u, int v) {
        return d[u] + d[v] - 2 * d[lca(u, v)];
    }
    auto path_sorted(int u, int v) {
        vector<tuple<int, int, bool>> U, V; // from, to, reversed
        while (head[u] != head[v]){
            if(d[head[u]] > d[head[v]]){
                U.emplace_back(tin[head[u]], tin[u], true);
                u = par[head[u]];
            }else{
                V.emplace_back(tin[head[v]], tin[v], false);
                v = par[head[v]];
            }
        }
        if(d[u] < d[v]) V.emplace_back(tin[u], tin[v], false);
        else U.emplace_back(tin[v], tin[u], true);
        U.insert(U.end(), V.rbegin(), V.rend());
        return U;
    }
};

void SOLVE(){
}
```

HopcroftKarp.cpp

```
#define ll long long
#define endl "\n"

struct HopcroftKarp {
    static const int inf = 1e9;
    int n;
    vector<int> l, r, d;
    vector<vector<int>> g;
    HopcroftKarp(int _n, int _m) {
        n = _n;
        int p = _n + _m + 1;
        g.resize(p); l.resize(p, 0); r.resize(p, 0); d.resize(p, 0);
    }
    void add_edge(int u, int v) {
        g[u].push_back(v + n);
    }
    bool bfs() {
        queue<int> q;
        for (int u = 1; u <= n; u++) {
            if (!l[u]) d[u] = 0, q.push(u);
            else d[u] = inf;
        }
        d[0] = inf;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto v : g[u]) {
                if (d[r[v]] == inf) { d[r[v]] = d[u] + 1; q.push(r[v]); }
            }
        }
        return d[0] != inf;
    }
    bool dfs(int u) {
        if (!u) return true;
        for (auto v : g[u]) {
            if (d[r[v]] == d[u] + 1 && dfs(r[v])) {
                l[u] = v; r[v] = u; return true;
            }
        }
        d[u] = inf;
        return false;
    }
    int maximum_matching() {
        int ans = 0;
```

```

    while (bfs()) {
        for(int u = 1; u <= n; u++) if (!l[u] && dfs(u)) ans++;
    }
    return ans;
}
};

void SOLVE(){
}

```

LCA and Binary lifting.cpp

```

struct LCA{
    int n, Log, timer;
    vector<vector<int>>> up;
    vector<int> depth, tin, tout;
    vector<ll> distRoot;
    LCA(){
        LCA(vector<vector<int>>> &g, int root = 0){
            init(g, root);
        }
        void init(vector<vector<int>>> &g, int root = 0){
            n = g.size();
            Log = __lg(n) + 1;
            timer = 0;
            up.assign(n, vector<int>(Log, root));
            depth.assign(n, 0);
            tin.assign(n, 0);
            tout.assign(n, 0);
            distRoot.assign(n, 0);
            dfs(root, root, g);
        }
        void dfs(int u, int p, vector<vector<int>>> &g){
            tin[u] = ++timer;
            up[u][0] = p;
            for (int l = 1; l < Log; l++){
                up[u][l] = up[up[u][l-1]][l-1];
            }
            for (int v : g[u]){
                if (v == p) continue;
                depth[v] = depth[u] + 1;
                distRoot[v] = distRoot[u] + 1;
                dfs(v, u, g);
            }
            tout[u] = timer;
        }
        int kth(int u, int k){
            while (k){
                u = up[u][__builtin_ctz(k)];
                k &= k - 1;
            }
            return u;
        }
        int jump(int u, int k){
            return kth(u, k);
        }
        bool isAnc(int u, int v){
            return tin[u] <= tin[v] && tout[u] <= tout[v];
        }
        int lca(int u, int v){
            if (depth[u] < depth[v]) swap(u, v);
            u = kth(u, depth[u] - depth[v]);
            if (u == v) return u;
            for (int l = Log - 1; l >= 0; l--){
                if (up[u][l] != up[v][l]){
                    u = up[u][l];
                    v = up[v][l];
                }
            }
            return up[u][0];
        }
        int dist(int u, int v, int l = -1){
            if (l == -1) l = lca(u, v);
            return depth[u] + depth[v] - 2 * depth[l];
        }
        ll distance(int u, int v, int l = -1){
            if (l == -1) l = lca(u, v);
            return distRoot[u] + distRoot[v] - 2 * distRoot[l];
        }
        int go(int u, int v, int k){
            int l = lca(u, v);
            int d = dist(u, v, l);
            assert(k <= d);
            if (depth[l] + k <= depth[u]) return kth(u, k);
            k = depth[u] - depth[l];
            return kth(v, depth[v] - depth[l] - k);
        }
    };

    void SOLVE(){
}

```

MCMF.cpp

```

const int inf = 1000000010;
struct Edge {
    int to, cost, cap, flow, backEdge;
};
struct MCMF {
    int s, t, n;
    vector<vector<Edge>>> g;
    MCMF(int _n, int _s, int _t){
        s = _s, t = _t, n = _n + 1;
        g = vector<vector<Edge>>>(n);
    }
    void addEdge(int u, int v, int cost, int cap) {
        g[u].push_back({v, cost, cap, 0, (int)g[v].size()});
        g[v].push_back({u, -cost, 0, 0, (int)g[u].size() - 1});
    }
    pair<int, int> minCostMaxFlow(int K) {
}

```

```

int flow = 0, cost = 0;
vector<int> state(n), from(n), from_edge(n), d(n);
deque<int> q;

while(flow < K){
    for (int i = 0; i < n; i++){
        state[i] = 2, d[i] = inf, from[i] = -1;
        state[s] = 1; q.clear(); q.push_back(s); d[s] = 0;
        while (!q.empty()) {
            int v = q.front(); q.pop_front(); state[v] = 0;
            for (int i = 0; i < (int) g[v].size(); i++) {
                Edge e = g[v][i];
                if (e.flow >= e.cap || d[e.to] <= d[v] + e.cost)
                    continue;
                int to = e.to; d[to] = d[v] + e.cost;
                from[to] = v; from_edge[to] = i;
                if(state[to] == 1) continue;
                q.push_back(to);
                state[to] = 1;
            }
        }
        if (d[t] == inf) break;
        int it = t, addflow = inf;
        while (it != s) {
            addflow = min(addflow, g[from[it]][from_edge[it]].cap - g[from[it]][from_edge[it]].flow);
            it = from[it];
        }
        addflow = min(addflow, K - flow);
        it = t;
        while (it != s) {
            g[from[it]][from_edge[it]].flow += addflow;
            g[it][g[from[it]][from_edge[it]].backEdge].flow -= addflow;
            cost += g[from[it]][from_edge[it]].cost * addflow;
            it = from[it];
        }
        flow += addflow;
    }
    return {flow, cost};
}

vector<vector<int>>> get_paths(int k) {
    vector<vector<int>>> paths;
    for(int i = 0; i < k; i++){
        vector<int> path;
        int curr = s;
        path.push_back(curr);
        while(curr != t){
            for(auto &e : g[curr]){
                if(e.cost == 1 && e.flow > 0){
                    e.flow--;
                    curr = e.to;
                    path.push_back(curr);
                    break;
                }
            }
        }
        paths.push_back(path);
    }
    return paths;
}

void SOLVE() {
}

```

Rerooting.cpp

```

const int N = 2e5 + 5;
using Agg = ll; // in-progress dp of a node
using Val = ll; // final dp of a node
vector<vector<int>>> graph; // 0-based nodes
int a[N];
// Initialize the DP state for a node
Agg base(int node) {
    return 0;
}
// Merge a child's finalized value into the current node's aggregate
Agg merge_into(Agg cur_dp, Val ch_val, int node, int edge_idx) {
    return cur_dp + ch_val;
}
// Finalize the aggregate before passing it up (add node/edge weights)
Val finalize_merge(Agg cur_dp, int node, int par_edge_idx) {
    return cur_dp;
}
// root_dp[u]: dp of the tree when rooted at u
// edge_dp[u][i]: dp of the subtree of node graph[u][i] excluding the branch of u
// rege_dp[u][i]: dp of the tree rooted at u excluding the branch of graph[u][i]
auto solve_reroot() {
    int n = graph.size();
    vector<Val> root_dp(n), dp(n);
    vector<vector<Val>>> edge_dp(n), rege_dp(n);
    vector<int> bfs, par(n, -1);
    bfs.reserve(n);
    bfs.push_back(0);
    for(int i = 0; i < n; i++){
        int u = bfs[i];
        for(int v : graph[u]){
            if(v == par[u]) continue;
            par[v] = u;
            bfs.push_back(v);
        }
    }
    for(int i = n - 1; i >= 0; i--){
        int u = bfs[i], p_idx = -1;
        Agg agg = base(u);
        for(int j = 0; j < (int)graph[u].size(); j++){
            int v = graph[u][j];
            if(v == par[u]) p_idx = j;
            else agg = merge_into(agg, dp[v], u, j);
        }
        dp[u] = finalize_merge(agg, u, p_idx);
    }
}

```

```

for(int u : bfs){
    int deg = graph[u].size();
    edge_dp[u].resize(deg);
    redge_dp[u].resize(deg);
    Agg tot = base(u);
    for (int j = 0; j < deg; j++){
        int v = graph[u][j];
        edge_dp[u][j] = (v == par[u] ? dp[u] : dp[v]);
        tot = merge_into(tot, edge_dp[u][j], u, j);
    }
    root_dp[u] = finalize_merge(tot, u, -1);
    if(deg) {
        struct Frame { int l, r; Agg out; };
        vector<Frame> st;
        st.reserve(deg * 2);
        st.push_back({0, deg - 1, base(u)});
        while(!st.empty()) {
            auto [l, r, out] = st.back();
            st.pop_back();
            if(l == r){
                redge_dp[u][l] = finalize_merge(out, u, l);
                if(graph[u][l] != par[u]) dp[graph[u][l]] = redge_dp[u][l];
                continue;
            }
            int mid = l + ((r - l) >> 1);
            Agg L_out = out, R_out = out;
            for(int i = mid + 1; i <= r; i++) L_out = merge_into(L_out,
                ↳ edge_dp[u][i], u, i);
            for(int i = l; i <= mid; i++) R_out = merge_into(R_out,
                ↳ edge_dp[u][i], u, i);
            st.push_back({mid + 1, r, R_out});
            st.push_back({l, mid, L_out});
        }
    }
    return make_tuple(root_dp, edge_dp, redge_dp);
}

void SOLVE(){
}

```

SCC.cpp

```

struct SCC{
    int n, components;
    vector<vector<int>> graph, rev, groups, dag;
    vector<int> comp, order, vis;
    SCC(){
        SCC(int _n){
            init(_n);
        }
        void init(int _n){
            n = _n;
            components = 0;
            graph.assign(n + 1, {});
            rev.assign(n + 1, {});
            comp.assign(n + 1, -1);
            vis.assign(n + 1, 0);
            order.clear();
            groups.clear();
            dag.clear();
        }
        void add_edge(int u, int v){
            graph[u].push_back(v);
            rev[v].push_back(u);
        }
        void dfs1(int v){
            vis[v] = 1;
            for (auto u : graph[v]){
                if (!vis[u]) dfs1(u);
            }
            order.push_back(v);
        }
        void dfs2(int v, int id){
            comp[v] = id;
            groups[id].push_back(v);
            for (auto u : rev[v]){
                if (comp[u] == -1) dfs2(u, id);
            }
        }
        void build(){
            fill(vis.begin(), vis.end(), 0);
            fill(comp.begin(), comp.end(), -1);
            order.clear();
            for (int i = 1; i <= n; i++){
                if (!vis[i]) dfs1(i);
            }
            reverse(order.begin(), order.end());
            components = 0;
            groups.clear();
            for (auto v : order){
                if (comp[v] == -1){
                    groups.push_back({});
                    dfs2(v, components++);
                }
            }
            dag.assign(components, {});
            vector<pair<int, int>> edges;
            for (int v = 1; v <= n; v++){
                for (auto u : graph[v]){
                    if (comp[v] != comp[u]) edges.push_back({comp[v], comp[u]});
                }
            }
            sort(edges.begin(), edges.end());
            edges.erase(unique(edges.begin(), edges.end(), edges.end()));
            for (auto [u, v] : edges){
                dag[u].push_back(v);
            }
        }
    };
};

void SOLVE(){
}

```

```

}

```

tree isomorphism.cpp

```

map<vector<int>, int> ids;
int ID = 1;

int hash_node(vector<int> ch){
    sort(ch.begin(), ch.end());
    if (!ids.count(ch)) ids[ch] = ID++;
    return ids[ch];
}

int rooted_hash(int v, int p, vector<vector<int>> &graph){
    vector<int> ch;
    for (auto u : graph[v]){
        if (u != p) ch.push_back(rooted_hash(u, v, graph));
    }
    return hash_node(ch);
}

vector<int> centers(vector<vector<int>> &graph){
    int n = graph.size() - 1;
    vector<int> deg(n + 1);
    queue<int> q;
    for (int i = 1; i <= n; i++){
        deg[i] = graph[i].size();
        if (deg[i] <= 1) q.push(i);
    }
    int left = n;
    while (left > 2){
        int sz = q.size();
        left -= sz;
        while (sz--){
            int v = q.front();
            q.pop();
            for (auto u : graph[v]){
                if (--deg[u] == 1) q.push(u);
            }
        }
    }
    vector<int> ret;
    while (!q.empty()){
        ret.push_back(q.front());
        q.pop();
    }
    return ret;
}

vector<int> tree_hash(vector<vector<int>> &graph){
    vector<int> ret;
    for (auto c : centers(graph)){
        ret.push_back(rooted_hash(c, 0, graph));
    }
    sort(ret.begin(), ret.end());
    return ret;
}

bool isomorphic(vector<vector<int>> &a, vector<vector<int>> &b){
    if (a.size() != b.size()) return false;
    return tree_hash(a) == tree_hash(b);
}

void SOLVE(){
}

```

Virtual Tree.cpp

```

struct LCA{
    int n, Log;
    vector<vector<int>> up;
    vector<int> depth, tin, tout;
    LCA(vector<vector<int>> &g, int root = 0) :
        n((int)g.size()), Log(__lg(n) + 1), up(n, vector<int>(Log, root)),
        ↳ depth(n), tin(n), tout(n) {
        int timer = 0;
        function<void(int)> dfs = [&](int u){
            tin[u] = ++timer;
            for (int v : g[u]){
                if (v == up[u][0]) continue;
                depth[v] = depth[u] + 1;
                up[v][0] = u;
                for (int l = 1; l < Log; l++){
                    up[v][l] = up[up[v][l - 1]][l - 1];
                }
                dfs(v);
            }
            tout[u] = timer;
        };
        dfs(root);
    }
    int kth(int u, int k){
        while (k){
            u = up[u][__builtin_ctz(k)];
            k &= k - 1;
        }
        return u;
    }
    bool isAnc(int u, int v){
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }
    int lca(int u, int v){
        if (depth[u] < depth[v]) swap(u, v);
        u = kth(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int l = Log - 1; l >= 0; l--){
            if (up[u][l] != up[v][l]){
                u = up[u][l];
                v = up[v][l];
            }
        }
        return up[u][0];
    }
    int dist(int u, int v){

```

```

        int l = lca(u, v);
        return depth[u] + depth[v] - 2 * depth[l];
    }
};

struct VirtualTree{
    vector<int> nodes;
    vector<tuple<int, int, int>> edges; // u, v, distance
    VirtualTree(vector<int> subset, LCA &lca){
        if (subset.empty()) return;
        auto cmp = [&](int u, int v){
            return lca.tin[u] < lca.tin[v];
        };
        sort(subset.begin(), subset.end(), cmp);
        int k = subset.size();
        for (int i = 0; i + 1 < k; i++){
            subset.push_back(lca.lca(subset[i], subset[i + 1]));
        }
        sort(subset.begin(), subset.end(), cmp);
        subset.erase(unique(subset.begin(), subset.end()), subset.end());
        nodes = subset;
        vector<int> st;
        st.push_back(subset[0]);
        for (int i = 1; i < (int)subset.size(); i++){
            int u = subset[i];
            while (!st.empty() && !lca.isAnc(st.back(), u)) st.pop_back();
            edges.push_back({st.back(), u, lca.dist(st.back(), u)});
            st.push_back(u);
        }
    }
};

void SOLVE(){
}

```

4 Math temps

modnum helper

Note: Shared Mint / modnum helper. Files below that say this is required keep their own MOD and using mint lines.

```
template<int MOD> struct modnum {
    int v;
    modnum(ll x = 0) : v(x % MOD) { if(v < 0) v += MOD; }
    explicit operator int() const { return v; }
    friend ostream& operator<<(ostream& out, const modnum& n) { return out <<
        <= n.v; }
    friend istream& operator>>(istream& in, modnum& n) { ll x; in >> x; n =
        <= modnum(x); return in; }
    friend bool operator==(const modnum& a, const modnum& b) { return a.v ==
        <= b.v; }
    friend bool operator!=(const modnum& a, const modnum& b) { return a.v !=
        <= b.v; }
    modnum pow(ll b) const {
        modnum a = *this, r = 1; while(b) { if(b & 1) r *= a; a *= a; b >>= 1; }
        return r;
    }
    modnum inv() const { return pow(MOD - 2); }
    modnum operator-() const { return modnum(v ? MOD - v : 0); }
    modnum& operator+=(const modnum& o) { v += o.v; if(v >= MOD) v -= MOD;
        <= return *this; }
    modnum& operator-=(const modnum& o) { v -= o.v; if(v < 0) v += MOD; return
        <= *this; }
    modnum& operator*=(const modnum& o) { v = int(ll(v) * o.v % MOD); return
        <= *this; }
    modnum& operator/=(const modnum& o) { return *this *= o.inv(); }
    friend modnum operator+(modnum a, const modnum& b) { return a += b; }
    friend modnum operator-(modnum a, const modnum& b) { return a -= b; }
    friend modnum operator*(modnum a, const modnum& b) { return a *= b; }
    friend modnum operator/(modnum a, const modnum& b) { return a /= b; }
};
```

BerlekampMassey.cpp

Note: Requires the Mint / modnum helper above.

```
const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){ if (b & 1) r *= a; a *= a; b >>= 1; }
    return r;
}

vector<mint> BerlekampMassey(vector<mint> S){
    int n = (int)S.size(), L = 0, m = 0;
    vector<mint> C(n), B(n), T;
    C[0] = B[0] = 1;
    mint b = 1;
    for (int i = 0; i < n; i++){
        ++m;
        mint d = S[i];
        for (int j = 1; j <= L; j++) d += C[j] * S[i - j];
        if (d == 0) continue;
        T = C;
        mint coef = d / b;
        for (int j = m; j < n; j++) C[j] -= coef * B[j - m];
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }
    C.resize(L + 1);
    C.erase(C.begin());
    for (auto &x : C) x = -x;
    return C;
}

vector<mint> combine(int n, vector<mint> &a, vector<mint> &b, vector<mint> &tr){
    vector<mint> res(n * 2 + 1, 0);
    for (int i = 0; i < n + 1; i++){
        for (int j = 0; j < n + 1; j++) res[i + j] += a[i] * b[j];
    }
    for (int i = 2 * n; i > n; i--){
        for (int j = 0; j < n; j++) res[i - 1 - j] += res[i] * tr[j];
    }
    res.resize(n + 1);
    return res;
}

mint LinearRecurrence(vector<mint> &S, vector<mint> &tr, ll k){
    int n = S.size();
    assert(n == (int)tr.size());
    if (n == 0) return 0;
    if (k < n) return S[k];
    vector<mint> pol(n + 1, e(n + 1));
    pol[0] = e[1] = 1;
    for (+k; k; k >>= 1){
        if (k & 1) pol = combine(n, pol, e, tr);
        e = combine(n, e, e, tr);
    }
    mint res = 0;
    for (int i = 0; i < n; i++) res += pol[i + 1] * S[i];
    return res;
}

bool guess(vector<mint> S, ll k, mint &ans){
    vector<mint> tr = BerlekampMassey(S);
    int d = tr.size();
    if (d == 0){
        ans = 0;
        return true;
    }
    if ((int)S.size() < 2 * d) return false;
    for (int i = d; i < (int)S.size(); i++){
        mint cur = 0;
        for (int j = 0; j < d; j++){
            cur += tr[j] * S[i - j - 1];
        }
        if (cur != S[i]) return false;
    }
```

```
    }
    vector<mint> init(S.begin(), S.begin() + d);
    ans = LinearRecurrence(init, tr, k);
    return true;
}

void SOLVE(){
}
```

comb.cpp

Note: Requires the Mint / modnum helper above.

```
#include<cassert>
const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){ if (b & 1) r *= a; a *= a; b >>= 1; }
    return r;
}

template<int N>
struct counting{
    mint fact[N], invfact[N], invfact[N];
    counting(){
        fact[0] = invfact[0] = 1;
        for (int i = 1; i < N; i++) fact[i] = fact[i - 1] * i;
        invfact[N - 1] = fact[N - 1].inv();
        for (int i = N - 2; i >= 0; i--) invfact[i] = invfact[i + 1] * (i + 1);
        for (int i = 1; i < N; i++) invfact[i] = fact[i - 1] * invfact[i];
    }
    mint choose(int n, int r){
        if (n < 0 || r < 0 || n < r) return 0;
        return fact[n] * invfact[n - r] * invfact[r];
    }
    mint stars_andBars(int n, int k){
        return choose(n + k - 1, k - 1);
    }
    mint P(int n, int r){
        if (n < 0 || r < 0 || r > n) return 0;
        return fact[n] * invfact[n - r];
    }
    mint catalan(int n){
        return fact[2 * n] * invfact[n] * invfact[n + 1];
    }
};

const int N = 1e6 + 5;
counting<N> comb;

void SOLVE(){
}
```

CRT.cpp

```
using T = __int128;

T norm(T a, T m){
    a %= m;
    if (a < 0) a += m;
    return a;
}

T exgcd(T a, T b, T &x, T &y){
    T xx = y = 0;
    T yy = x = 1;
    while (b){
        T q = a / b, t = b;
        b = a % b; a = t;
        t = xx;
        xx = x - q * xx;
        x = t; t = yy;
        yy = y - q * yy;
        y = t;
    }
    return a;
}

pair<T, T> CRT(T a1, T m1, T a2, T m2){
    a1 = norm(a1, m1);
    a2 = norm(a2, m2);
    T p, q;
    T g = exgcd(m1, m2, p, q);
    if (a1 % g != a2 % g) return {0, -1};
    T m = m1 / g * m2;
    p = norm(p, m);
    q = norm(q, m);
    T x = (p * a2 % m * (m1 / g) % m + q * a1 % m * (m2 / g) % m) % m;
    return {x, m};
}

pair<T, T> CRT(vector<pair<T, T>> eq){
    pair<T, T> ret = {0, 1};
    for (auto [a, m] : eq){
        ret = CRT(ret.first, ret.second, a, m);
        if (ret.second == -1) return ret;
    }
    return ret;
}

pair<T, T> intersect(T a1, T d1, T a2, T d2){
    auto x = CRT(a1, d1, a2, d2);
    T a = x.first, d = x.second;
    if (d == -1) return {0, 0};
    T st = max(a1, a2);
    if (a < st) a += (st - a + d - 1) / d * d;
    return {a, d};
}

template<typename T>
T mod_inv_in_range(T a, T m){
}
```

```

T x = a, y = m;
T vx = 1, vy = 0;
while (x){
    T k = y / x;
    y %= x;
    vy -= k * vx;
    swap(x, y);
    swap(vx, vy);
}
assert(y == 1);
return vy < 0 ? m + vy : vy;
}
template<typename T>
struct extended_gcd_result{
    T gcd, coeff_a, coeff_b;
};
template<typename T>
extended_gcd_result<T> extended_gcd(T a, T b){
    T x = a, y = b;
    T ax = 1, ay = 0;
    T bx = 0, by = 1;
    while (x){
        T k = y / x;
        y %= x;
        ay -= k * ax;
        by -= k * bx;
        swap(x, y);
        swap(ax, ay);
        swap(bx, by);
    }
    return {y, ay, by};
}
template<typename T>
T mod_inv(T a, T m){
    a %= m;
    if (a < 0) a += m;
    return mod_inv_in_range(a, m);
}

void SOLVE(){
}

```

Discrete log.cpp

```

ll mul_mod(ll a, ll b, ll mod){
    return (__int128)a * b % mod;
}
ll fp(ll a, ll b, ll mod){
    a %= mod;
    ll res = 1 % mod;
    while (b){
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}
// returns minimum x >= 0 with a^x = b (mod mod), or -1 if none
ll discrete_log(ll a, ll b, ll mod){
    a %= mod;
    b %= mod;
    if (mod == 1) return 0;
    ll add = 0, k = 1, g;
    while ((g = gcd(a, mod)) > 1){
        if (b == k) return add;
        if (b % g) return -1;
        b /= g;
        mod /= g;
        add++;
        k = mul_mod(k, a / g, mod);
    }
    if (b == k) return add;
    ll n = sqrtl(mod) + 1;
    unordered_map<ll, ll> vals;
    ll cur = b;
    for (ll q = 0; q <= n; q++){
        vals[cur] = q;
        cur = mul_mod(cur, a, mod);
    }
    ll an = fp(a, n, mod);
    cur = k;
    for (ll p = 1; p <= n + 1; p++){
        cur = mul_mod(cur, an, mod);
        auto it = vals.find(cur);
        if (it != vals.end()){
            ll ans = n * p - it->second + add;
            if (ans >= 0) return ans;
        }
    }
    return -1;
}

void SOLVE(){
}

```

FFT string_matching.cpp

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> &a){
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <= 1){
        rt.resize(n);
        double ang = PI / k;
        for (int i = k; i < 2 * k; i++){
            rt[i] = polar(1.0, ang * (i - k));
        }
    }
}

```

```

vector<int> rev(n);
for (int i = 0; i < n; i++){
    rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
}
for (int i = 0; i < n; i++){
    if (i < rev[i]) swap(a[i], a[rev[i]]);
}
for (int k = 1; k < n; k <= 1){
    for (int i = 0; i < n; i += 2 * k){
        for (int j = 0; j < k; j++){
            cd z = rt[j + k] * a[i + j + k], &ai = a[i + j];
            a[i + j + k] = ai - z;
            ai += z;
        }
    }
}
vector<ll> conv(const vector<ll> &a, const vector<ll> &b){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(s), n = 1 << B;
    vector<cd> L(n), R(n), out(n);
    for (int i = 0; i < (int)a.size(); i++) L[i] = a[i];
    for (int i = 0; i < (int)b.size(); i++) R[i] = b[i];
    fft(L);
    fft(R);
    for (int i = 0; i < n; i++){
        out[-i & (n - 1)] = L[i] * R[i] / (double)n;
    }
    fft(out);
    vector<ll> ret(s);
    for (int i = 0; i < s; i++){
        ret[i] = llround(real(out[i]));
    }
    return ret;
}

// match iff sum s*t*(s-t)^2 over aligned non-'?' chars is 0
vector<int> wildcard_matching(const string &s, const string &t){
    int n = s.size(), m = t.size();
    if (m > n) return {};
    vector<ll> s1(n), s2(n), s3(n), t1(m), t2(m), t3(m);
    for (int i = 0; i < n; i++){
        s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1;
        s2[i] = s1[i] * s1[i];
        s3[i] = s1[i] * s2[i];
    }
    for (int i = 0; i < m; i++){
        t1[i] = t[i] == '?' ? 0 : t[i] - 'a' + 1;
        t2[i] = t1[i] * t1[i];
        t3[i] = t1[i] * t2[i];
    }
    reverse(t1.begin(), t1.end());
    reverse(t2.begin(), t2.end());
    reverse(t3.begin(), t3.end());
    vector<ll> a = conv(s1, t3);
    vector<ll> b = conv(s2, t2);
    vector<ll> c = conv(s3, t1);
    vector<int> ret;
    for (int i = m - 1; i < n; i++){
        if (a[i] - 2 * b[i] + c[i] == 0) ret.push_back(i - m + 1);
    }
    return ret;
}

void SOLVE(){
}

```

FFT with MOD.cpp

Note: Requires the Mint / modnum helper above.

```

const int MOD = 1e9 + 7;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){
        if (b & 1) r *= a; a *= a; b >>= 1;
    }
    return r;
}

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> &a){
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <= 1){
        rt.resize(n);
        double ang = PI / k;
        for (int i = k; i < 2 * k; i++){
            rt[i] = polar(1.0, ang * (i - k));
        }
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++){
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    }
    for (int i = 0; i < n; i++){
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int k = 1; k < n; k <= 1){
        for (int i = 0; i < n; i += 2 * k){
            for (int j = 0; j < k; j++){
                cd z = rt[j + k] * a[i + j + k], &ai = a[i + j];
                a[i + j + k] = ai - z;
                ai += z;
            }
        }
    }
}

```

```

    }
}
vector<ll> conv_mod(const vector<ll> &a, const vector<ll> &b, ll mod = MOD){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(s), n = 1 << B;
    int cut = sqrt(mod);
    vector<cd> L(n), R(n), outl(n), outs(n);
    for (int i = 0; i < (int)a.size(); i++){
        ll x = a[i] % mod;
        if (x < 0) x += mod;
        L[i] = cd(x / cut, x % cut);
    }
    for (int i = 0; i < (int)b.size(); i++){
        ll x = b[i] % mod;
        if (x < 0) x += mod;
        R[i] = cd(x / cut, x % cut);
    }
    fft(L);
    fft(R);
    cd I(0, 1);
    for (int i = 0; i < n; i++){
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / I;
    }
    fft(outl);
    fft(outs);
    vector<ll> res(s);
    for (int i = 0; i < s; i++){
        ll av = ll(real(outl[i]) + 0.5);
        ll cv = ll(imag(outs[i]) + 0.5);
        ll bv = ll(imag(outl[i]) + 0.5) + ll(real(outs[i]) + 0.5);
        res[i] = ((av % mod * cut + bv) % mod * cut + cv) % mod;
        if (res[i] < 0) res[i] += mod;
    }
    return res;
}
vector<mint> conv(const vector<mint> &a, const vector<mint> &b){
    if (a.empty() || b.empty()) return {};
    vector<ll> A(a.size()), B(b.size());
    for (int i = 0; i < (int)a.size(); i++) A[i] = a[i].v;
    for (int i = 0; i < (int)b.size(); i++) B[i] = b[i].v;
    auto c = conv_mod(A, B, MOD);
    vector<mint> ret(c.size());
    for (int i = 0; i < (int)c.size(); i++) ret[i] = c[i];
    return ret;
}

void SOLVE(){
}

```

FFT.cpp

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> &a){
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <= 1){
        rt.resize(n);
        double ang = PI / k;
        for (int i = k; i < 2 * k; i++){
            rt[i] = polar(1.0, ang * (i - k));
        }
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++){
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    }
    for (int i = 0; i < n; i++){
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int k = 1; k < n; k <= 1){
        for (int i = 0; i < n; i += 2 * k){
            for (int j = 0; j < k; j++){
                cd z = rt[j + k] * a[i + j + k], &ai = a[i + j];
                a[i + j + k] = ai - z;
                ai += z;
            }
        }
    }
}

vector<ll> conv(const vector<ll> &a, const vector<ll> &b){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(s), n = 1 << B;
    vector<cd> L(n), R(n), out(n);
    for (int i = 0; i < (int)a.size(); i++) L[i] = a[i];
    for (int i = 0; i < (int)b.size(); i++) R[i] = b[i];
    fft(L);
    fft(R);
    for (int i = 0; i < n; i++){
        out[-i & (n - 1)] = L[i] * R[i] / (double)n;
    }
    fft(out);
    vector<ll> ret(s);
    for (int i = 0; i < s; i++){
        ret[i] = llround(real(out[i]));
    }
    return ret;
}

void SOLVE(){
}

```

FWHT.cpp

Note: Requires the Mint / modnum helper above.

```

const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){
        if (b & 1) r *= a; a *= a; b >>= 1;
    }
    return r;
}

bool is_power_two(int n){
    return n > 0 && (n & (n - 1)) == 0;
}

template<typename T>
void xor_transform(vector<T> &a, bool inv = false){
    int n = a.size();
    assert(is_power_two(n));
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                T x = a[i + j], y = a[i + j + len];
                a[i + j] = x + y;
                a[i + j + len] = x - y;
            }
        }
    }
    if (inv){
        T iv = T(n);
        for (auto &x : a) x /= iv;
    }
}

template<typename T>
void and_transform(vector<T> &a, bool inv = false){
    int n = a.size();
    assert(is_power_two(n));
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                if (inv) a[i + j] -= a[i + j + len];
                else a[i + j] += a[i + j + len];
            }
        }
    }
}

template<typename T>
void or_transform(vector<T> &a, bool inv = false){
    int n = a.size();
    assert(is_power_two(n));
    for (int len = 1; len < n; len <= 1){
        for (int i = 0; i < n; i += 2 * len){
            for (int j = 0; j < len; j++){
                if (inv) a[i + j + len] -= a[i + j];
                else a[i + j + len] += a[i + j];
            }
        }
    }
}

template<typename T>
vector<T> xor_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    xor_transform(a);
    xor_transform(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    xor_transform(a, true);
    return a;
}

template<typename T>
vector<T> and_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    and_transform(a);
    and_transform(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    and_transform(a, true);
    return a;
}

template<typename T>
vector<T> or_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    or_transform(a);
    or_transform(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    or_transform(a, true);
    return a;
}

template<typename T>
vector<T> subset_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size() && is_power_two(a.size()));
    int n = a.size(), lg = __builtin_ctz(n);
    vector<vector<T>> A(lg + 1, vector<T>(n)), B(lg + 1, vector<T>(n));
    for (int mask = 0; mask < n; mask++){
        A[__builtin_popcount(mask)][mask] = a[mask];
        B[__builtin_popcount(mask)][mask] = b[mask];
    }
    for (int i = 0; i <= lg; i++){
        or_transform(A[i]);
        or_transform(B[i]);
    }
    vector<T> tmp(n), ret(n);
    for (int k = 0; k <= lg; k++){
        fill(tmp.begin(), tmp.end(), T(0));
        for (int i = 0; i <= k; i++){
            for (int mask = 0; mask < n; mask++){
                tmp[mask] += A[i][mask] * B[k - i][mask];
            }
        }
        or_transform(tmp, true);
        for (int mask = 0; mask < n; mask++){
            if (__builtin_popcount(mask) == k) ret[mask] = tmp[mask];
        }
    }
}

```

```

    }
    return ret;
}

void SOLVE(){
}

```

Gaussian with Doubles.cpp

```

const double EPS = 1e-9;
const int INF = 2;
int gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = a.size(), m = (int)a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int sel = row;
        for (int i = row; i < n; i++) {
            if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        }
        if (abs(a[sel][col]) < EPS) continue;
        for (int j = col; j <= m; j++) swap(a[sel][j], a[row][j]);
        where[col] = row;

        for (int i = 0; i < n; i++) {
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j = col; j <= m; j++) {
                    a[i][j] -= a[row][j] * c;
                }
            }
        }
        row++;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++) {
        if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
    }
    for (int i = 0; i < n; i++) {
        double sum = 0;
        for (int j = 0; j < m; j++) sum += ans[j] * a[i][j];
        if (abs(sum - a[i][m]) > EPS) return 0;
    }
    for (int i = 0; i < m; i++) {
        if (where[i] == -1) return INF;
    }
    return 1;
}

void SOLVE() {
}

```

Gaussian with MOD.cpp

```

const ll MOD = 1e9 + 7;
int norm(ll x){
    x %= MOD;
    if (x < 0) x += MOD;
    return x;
}

ll binpow(ll a, ll b){
    ll res = 1;
    while(b){
        if(b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int Gauss(vector<vector<int>> a, vector<int> &ans){
    int n = a.size(), m = (int)a[0].size() - 1;
    vector<int> pos(m, -1);
    int free_var = 0;
    const ll MODSQ = (ll)MOD * MOD;
    int det = 1, rank = 0;
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int mx = row;
        for (int k = row; k < n; k++) if (a[k][col] > a[mx][col]) mx = k;
        if (a[mx][col] == 0) {det = 0; continue;}
        for (int j = col; j <= m; j++) swap(a[mx][j], a[row][j]);
        if (row != mx) det = det == 0 ? 0 : MOD - det;
        det = 1LL * det * a[row][col] % MOD;
        pos[col] = row;
        int inv = binpow(a[row][col], MOD - 2);
        for (int i = 0; i < n && inv; i++){
            if (i != row && a[i][col]) {
                int x = ((ll)a[i][col] * inv) % MOD;
                for (int j = col; j <= m && x; j++){
                    if (a[row][j]) a[i][j] = (MODSQ + a[i][j] - ((ll)a[row][j] * x)) % MOD;
                }
            }
        }
        row++; ++rank;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++){
        if (pos[i] == -1) free_var++;
        else ans[i] = ((ll)a[pos[i]][m] * binpow(a[pos[i]][i], MOD - 2)) % MOD;
    }
    for (int i = 0; i < n; i++) {
        ll val = 0;
        for (int j = 0; j < m; j++) val = (val + ((ll)ans[j] * a[i][j])) % MOD;
        if (val != a[i][m]) return -1; //no solution
    }
    return free_var; //has solution
}

int determinant(vector<vector<int>> a){
    int n = a.size();
    if (n == 0) return 1;
    assert((int)a[0].size() == n);
}

```

```

int det = 1;
for (int i = 0; i < n; i++){
    for (int j = 0; j < n; j++) a[i][j] = norm(a[i][j]);
}
for (int col = 0, row = 0; col < n; col++, row++){
    int sel = row;
    while (sel < n && a[sel][col] == 0) sel++;
    if (sel == n) return 0;
    if (sel != row){
        swap(a[sel], a[row]);
        det = norm(-det);
    }
    det = 1LL * det * a[row][col] % MOD;
    int inv = binpow(a[row][col], MOD - 2);
    for (int i = row + 1; i < n; i++){
        int c = 1LL * a[i][col] * inv % MOD;
        for (int j = col; j < n; j++){
            a[i][j] = norm(a[i][j] - 1LL * a[row][j] * c);
        }
    }
}
return det;
}

int matrix_rank(vector<vector<int>> a){
    int n = a.size();
    if (n == 0) return 0;
    int m = a[0].size(), rank = 0;
    for (int i = 0; i < n; i++){
        for (int j = 0; j < m; j++) a[i][j] = norm(a[i][j]);
    }
    for (int col = 0, row = 0; col < m && row < n; col++){
        int sel = row;
        while (sel < n && a[sel][col] == 0) sel++;
        if (sel == n) continue;
        swap(a[sel], a[row]);
        int inv = binpow(a[row][col], MOD - 2);
        for (int i = row + 1; i < n; i++){
            int c = 1LL * a[i][col] * inv % MOD;
            for (int j = col; j < m; j++){
                a[i][j] = norm(a[i][j] - 1LL * a[row][j] * c);
            }
        }
        row++;
        rank++;
    }
    return rank;
}

void SOLVE() {
}

```

lagrange interpolation.cpp

Note: Requires the Mint / modnum helper above.

```

const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){
        if (b & 1) r *= a; a *= a; b >>= 1;
    }
    return r;
}

// y[i] = f(first + i * step)
mint interpolate(const vector<mint> &y, ll x, mint first = 0, mint step = 1){
    int n = y.size() - 1;
    assert(n >= 0 && step != 0);
    if (first == 0 && step == 1 && 0 <= x && x <= n) return y[x];
    vector<mint> pref(n + 1, 1), suff(n + 1, 1), fact(n + 1, 1), finv(n + 1, 1);
    for (int i = 1; i <= n; i++){
        pref[i] = pref[i - 1] * (mint(x) - (first + mint(i - 1) * step));
        fact[i] = fact[i - 1] * step * i;
    }
    finv[n] = fact[n].inv();
    for (int i = n - 1; i >= 0; i--){
        suff[i] = suff[i + 1] * (mint(x) - (first + mint(i + 1) * step));
        finv[i] = finv[i + 1] * (i + 1) * step;
    }
    mint ans = 0;
    for (int i = 0; i <= n; i++){
        mint cur = y[i] * pref[i] * suff[i] * finv[i] * finv[n - i];
        if ((n - i) & 1) ans -= cur;
        else ans += cur;
    }
    return ans;
}

mint interpolate_brute(const vector<pair<mint, mint>> &p, mint x){
    int n = p.size();
    mint ans = 0;
    for (int i = 0; i < n; i++){
        mint num = 1, den = 1;
        for (int j = 0; j < n; j++){
            if (i == j) continue;
            num *= x - p[j].first;
            den *= p[i].first - p[j].first;
        }
        ans += p[i].second * num / den;
    }
    return ans;
}

mint sum_pows(ll n, int k){
    vector<mint> y(k + 2);
    for (int i = 1; i <= k + 1; i++){
        y[i] = y[i - 1] + binpow(i, k);
    }
    return interpolate(y, n);
}

```

```
void SOLVE(){
}
```

matrix_expo.cpp

Note: Requires the Mint / modnum helper above.

```
const int MOD = 998244353;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){
        if (b & 1) r *= a; a *= a; b >>= 1;
    }
    return r;
}

template<typename T>
struct Mat{
    int r, c;
    vector<vector<T>> a;
    Mat(int r = 0, int c = 0) : r(r), c(c), a(r, vector<T>(c)) {}
    static Mat iden(int n){
        Mat res(n, n);
        for (int i = 0; i < n; i++) res.a[i][i] = 1;
        return res;
    }
    vector<T> mul_vec(const vector<T> &v) const {
        assert(c == (int)v.size());
        vector<T> res(r);
        for (int i = 0; i < r; i++){
            for (int j = 0; j < c; j++){
                res[i] += a[i][j] * v[j];
            }
        }
        return res;
    }
};

template<typename T>
Mat<T> mul(const Mat<T> &a, const Mat<T> &b){
    assert(a.c == b.r);
    Mat<T> res(a.r, b.c);
    for (int i = 0; i < a.r; i++){
        for (int k = 0; k < a.c; k++){
            if (a.a[i][k] == T(0)) continue;
            for (int j = 0; j < b.c; j++){
                res.a[i][j] += a.a[i][k] * b.a[k][j];
            }
        }
    }
    return res;
}

template<typename T>
Mat<T> matpow(Mat<T> a, ll b){
    assert(a.r == a.c && b >= 0);
    Mat<T> res = Mat<T>::iden(a.r);
    while (b){
        if (b & 1) res = mul(res, a);
        a = mul(a, a);
        b >>= 1;
    }
    return res;
}

void SOLVE(){
}
```

NTT.cpp

Note: Requires the Mint / modnum helper above.

```
const int MOD = 998244353;
const int root = 3;
using mint = modnum<MOD>;
template<typename T>
mint binpow(T aa, ll b){
    assert(b >= 0);
    mint a = aa, r = 1;
    while (b){
        if (b & 1) r *= a; a *= a; b >>= 1;
    }
    return r;
}

int fp(int b, ll p){
    int res = 1;
    while (p){
        if (p & 1) res = 1LL * res * b % MOD;
        b = 1LL * b * b % MOD;
        p >>= 1;
    }
    return res;
}

void ntt(vector<int> &a){
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<int> rt(2, 1);
    for (static int k = 2, s = 2; k < n; k <= 1, s++){
        rt.resize(n);
        int z[] = {1, fp(root, MOD >> s)};
        for (int i = k; i < 2 * k; i++){
            rt[i] = 1LL * rt[i / 2] * z[i & 1] % MOD;
        }
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++){
```

```
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    }
    for (int i = 0; i < n; i++){
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int k = 1; k < n; k <= 1){
        for (int i = 0; i < n; i += 2 * k){
            for (int j = 0; j < k; j++){
                int z = 1LL * rt[j + k] * a[i + j + k] % MOD, &ai = a[i + j];
                a[i + j + k] = ai - z + (z > ai ? MOD : 0);
                ai += (ai + z >= MOD ? z - MOD : z);
            }
        }
    }
}

vector<mint> conv(const vector<mint> &a, const vector<mint> &b){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(s), n = 1 << B;
    int inv = fp(n, MOD - 2);
    vector<int> L(n), R(n), out(n);
    for (int i = 0; i < (int)a.size(); i++) L[i] = a[i].v;
    for (int i = 0; i < (int)b.size(); i++) R[i] = b[i].v;
    ntt(L);
    ntt(R);
    for (int i = 0; i < n; i++){
        out[-i & (n - 1)] = (1LL)L[i] * R[i] % MOD * inv % MOD;
    }
    ntt(out);
    vector<mint> ret(s);
    for (int i = 0; i < s; i++) ret[i] = out[i];
    return ret;
}

vector<mint> poly_trim(const vector<mint> &a, int sz){
    int n = min<int>(a.size(), sz);
    return vector<mint>(a.begin(), a.begin() + n);
}

vector<mint> poly_add(const vector<mint> &a, const vector<mint> &b, int sz = -1){
    if (sz == -1) sz = max<int>(a.size(), b.size());
    vector<mint> r(sz);
    for (int i = 0; i < sz; i++){
        if (i < (int)a.size()) r[i] += a[i];
        if (i < (int)b.size()) r[i] += b[i];
    }
    return r;
}

vector<mint> poly_sub(const vector<mint> &a, const vector<mint> &b, int sz = -1){
    if (sz == -1) sz = max<int>(a.size(), b.size());
    vector<mint> r(sz);
    for (int i = 0; i < sz; i++){
        if (i < (int)a.size()) r[i] += a[i];
        if (i < (int)b.size()) r[i] -= b[i];
    }
    return r;
}

vector<mint> poly_scalar_mul(const vector<mint> &a, mint scalar){
    vector<mint> r = a;
    for (auto &x : r) x *= scalar;
    return r;
}

vector<mint> poly_derivative(const vector<mint> &a){
    if (a.size() <= 1) return vector<mint>{0};
    vector<mint> d(a.size() - 1);
    for (int i = 1; i < (int)a.size(); i++) d[i - 1] = a[i] * i;
    return d;
}

vector<mint> poly_integral(const vector<mint> &a){
    int n = a.size();
    vector<mint> res(n + 1);
    for (int i = 0; i < n; i++) res[i + 1] = a[i] / (i + 1);
    return res;
}

vector<mint> poly_inv(const vector<mint> &F, int m){
    assert(!F.empty() && F[0] != 0);
    vector<mint> G(1, F[0].inv());
    int cur = 1;
    while (cur < m){
        cur <= 1;
        vector<mint> Fcut = poly_trim(F, cur);
        Fcut.resize(cur);
        vector<mint> GG = conv(conv(G, G), Fcut);
        G.resize(cur);
        for (int i = 0; i < cur; i++){
            mint u = i < (int)GG.size() ? GG[i] : 0;
            G[i] = G[i] * 2 - u;
        }
        G.resize(m);
        return G;
    }
}

vector<mint> poly_ln(const vector<mint> &F, int n){
    assert(!F.empty() && F[0] == 1);
    vector<mint> prod = conv(poly_derivative(F), poly_inv(F, n));
    prod.resize(max(0, n - 1));
    vector<mint> ret = poly_integral(prod);
    ret.resize(n);
    return ret;
}

vector<mint> poly_exp(const vector<mint> &A, int n){
    assert(!A.empty() && A[0] == 0);
    vector<mint> G(1, 1);
    int cur = 1;
    while (cur < n){
        cur <= 1;
        vector<mint> Gcut = poly_trim(G, cur);
        Gcut.resize(cur);
        vector<mint> diff = poly_sub(poly_trim(A, cur), poly_ln(Gcut, cur), cur);
        diff[0] += 1;
        G = conv(G, diff);
        G.resize(cur);
    }
    G.resize(n);
    return G;
}
```

```

}
vector<mint> poly_pow(const vector<mint> &F, ll k, int n){
    if (n == 0) return {};
    if (k == 0){
        vector<mint> r(n);
        r[0] = 1;
        return r;
    }
    int shift = 0;
    while (shift < (int)F.size() && F[shift] == 0) shift++;
    if (shift == (int)F.size()) return vector<mint>(n, 0);
    if (1LL * shift * k >= n) return vector<mint>(n, 0);
    mint lead = F[shift];
    vector<mint> G;
    for (int i = shift; i < (int)F.size(); i++) G.push_back(F[i] / lead);
    G.resize(n);
    vector<mint> lnG = poly_ln(G, n);
    for (auto &x : lnG) x *= mint(k);
    vector<mint> H = poly_exp(lnG, n);
    mint coef = lead.pow(k);
    for (auto &x : H) x *= coef;
    vector<mint> res(n);
    for (int i = 0; i + shift * k < n && i < (int)H.size(); i++){
        res[i + shift * k] = H[i];
    }
    return res;
}
vector<mint> poly_pow2(vector<mint> poly, ll p, int limit = 1e5 + 5){
    vector<mint> ans{1};
    while (p){
        if (p & 1) ans = conv(ans, poly);
        poly = conv(poly, poly);
        ans.resize(limit);
        poly.resize(limit);
        p >>= 1;
    }
    return ans;
}
int mod_sqrt(int a, int p = MOD){
    a %= p;
    if (a < 0) a += p;
    if (a == 0) return 0;
    if (fp(a, (p - 1) / 2) != 1) return -1;
    int q = p - 1, m = 0;
    while ((q & 1) == 0){
        q >>= 1;
        m++;
    }
    int z = 2;
    while (fp(z, (p - 1) / 2) != p - 1) z++;
    int c = fp(z, q), t = fp(a, q), r = fp(a, (q + 1) / 2);
    while (m > 1){
        int tmp = t, i = 0;
        for (; i < m - 1 && tmp != 1; i++) tmp = (1ll)tmp * tmp % p;
        if (i == 0) break;
        int b = c;
        for (int j = 0; j < m - i - 2; j++) b = (1ll)b * b % p;
        r = (1ll)r * b % p;
        c = (1ll)b * b % p;
        t = (1ll)t * c % p;
        m = i;
    }
    return min(r, p - r);
}
vector<mint> poly_sqrt(const vector<mint> &F, int n){
    if (F.empty() || n == 0) return {};
    int shift = 0;
    while (shift < (int)F.size() && F[shift] == 0) shift++;
    if (shift == (int)F.size()) return vector<mint>(n, 0);
    if (shift & 1) return {};
    int c0 = mod_sqrt(F[shift].v);
    if (c0 == -1) return {};
    int target = n - shift / 2;
    vector<mint> shifted_F(F.begin() + shift, F.end());
    vector<mint> G(1, c0);
    mint inv2 = mint(2).inv();
    int cur = 1;
    while (cur < target){
        cur <<= 1;
        vector<mint> Fcut = poly_trim(shifted_F, cur);
        Fcut.resize(cur);
        vector<mint> term = conv(Fcut, poly_inv(G, cur));
        term.resize(cur);
        G.resize(cur);
        for (int i = 0; i < cur; i++) G[i] = (G[i] + term[i]) * inv2;
    }
    vector<mint> res(n);
    for (int i = 0; i < target && i + shift / 2 < n; i++){
        res[i + shift / 2] = G[i];
    }
    return res;
}
void SOLVE(){
}

```

NTT_CRT.cpp

```

constexpr int MOD1 = 998244353;
constexpr int MOD2 = 1004535809;
constexpr int MOD3 = 469762049;

template<int MOD, int root>
struct NTT{
    static int fp(int b, int p){
        int res = 1;
        while (p){
            if (p & 1) res = 1LL * res * b % MOD;
            b = 1LL * b * b % MOD;
            p >>= 1;
        }
        return res;
    }
}

```

```

}
static void ntt(vector<int> &a){
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<int> rt(2, 1);
    for (static int k = 2, s = 2; k < n; k <= 1, s++){
        rt.resize(n);
        int z[] = {1, fp(root, MOD >> s)};
        for (int i = k; i < 2 * k; i++){
            rt[i] = 1LL * rt[i / 2] * z[i & 1] % MOD;
        }
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++){
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    }
    for (int i = 0; i < n; i++){
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int k = 1; k < n; k <= 1){
        for (int i = 0; i < n; i += 2 * k){
            for (int j = 0; j < k; j++){
                int z = 1LL * rt[j + k] * a[i + j + k] % MOD, &ai = a[i + j];
                a[i + j + k] = ai - z + (z > ai ? MOD : 0);
                ai += (ai + z >= MOD ? z - MOD : z);
            }
        }
    }
}
static vector<int> conv(const vector<int> &a, const vector<int> &b){
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1;
    int B = 32 - __builtin_clz(s), n = 1 << B;
    int inv = fp(n, MOD - 2);
    vector<int> L(a), R(b), out(n);
    L.resize(n);
    R.resize(n);
    ntt(L);
    ntt(R);
    for (int i = 0; i < n; i++){
        out[-i & (n - 1)] = (1ll)L[i] * R[i] % MOD * inv % MOD;
    }
    ntt(out);
    return {out.begin(), out.begin() + s};
}
};

using NTT1 = NTT<MOD1, 3>;
using NTT2 = NTT<MOD2, 3>;
using NTT3 = NTT<MOD3, 3>;

ll extGCD(ll a, ll b, ll &x, ll &y){
    if (b == 0){
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll d = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
ll inv(ll a, ll m){
    ll x, y;
    extGCD(a, m, x, y);
    return (x % m + m) % m;
}
vector<array<ll, 3>> conv3(const vector<ll> &a, const vector<ll> &b){
    vector<int> a1(a.size()), a2(a.size()), a3(a.size());
    vector<int> b1(b.size()), b2(b.size()), b3(b.size());
    for (int i = 0; i < (int)a.size(); i++){
        a1[i] = (a[i] % MOD1 + MOD1) % MOD1;
        a2[i] = (a[i] % MOD2 + MOD2) % MOD2;
        a3[i] = (a[i] % MOD3 + MOD3) % MOD3;
    }
    for (int i = 0; i < (int)b.size(); i++){
        b1[i] = (b[i] % MOD1 + MOD1) % MOD1;
        b2[i] = (b[i] % MOD2 + MOD2) % MOD2;
        b3[i] = (b[i] % MOD3 + MOD3) % MOD3;
    }
    auto c1 = NTT1::conv(a1, b1);
    auto c2 = NTT2::conv(a2, b2);
    auto c3 = NTT3::conv(a3, b3);
    vector<array<ll, 3>> ret(c1.size());
    for (int i = 0; i < (int)c1.size(); i++){
        ret[i] = {c1[i], c2[i], c3[i]};
    }
    return ret;
}
vector<ll> conv_crt(const vector<ll> &a, const vector<ll> &b, ll target_mod = 0){
    if (a.empty() || b.empty()) return {};
    auto c = conv3(a, b);
    ll m1m2 = (1ll)MOD1 * MOD2;
    ll inv_m1_m2 = inv(m1m2, MOD2);
    ll inv_m1m2_m3 = inv(m1m2 % MOD3, MOD3);
    __int128_t M = (__int128_t)MOD1 * MOD2 * MOD3;
    vector<ll> res(c.size());
    for (int i = 0; i < (int)c.size(); i++){
        ll v1 = c[i][0], v2 = c[i][1], v3 = c[i][2];
        ll x = (v2 - v1) % MOD2;
        if (x < 0) x += MOD2;
        x = x * inv_m1_m2 % MOD2;
        ll val12 = v1 + x * MOD1;
        ll y = (v3 - val12 % MOD3) % MOD3;
        if (y < 0) y += MOD3;
        y = y * inv_m1m2_m3 % MOD3;
        __int128_t ans = val12 + (__int128_t)y * m1m2;
        if (target_mod){
            ans %= target_mod;
            if (ans < 0) ans += target_mod;
        } else if (ans > M / 2){
            ans -= M;
        }
    }
}

```

```

        res[i] = (1ll)ans;
    }
    return res;
}

void SOLVE(){
}

```

Number_theory.cpp

```

11 fp(1ll a, 1ll b, 1ll mod){
    a %= mod;
    if (a < 0) a += mod;
    1ll res = 1 % mod;
    while (b){
        if (b & 1) res = (__int128)res * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return res;
}

11 exgcd(1ll a, 1ll b, 1ll &x, 1ll &y){
    if (b == 0){
        x = 1;
        y = 0;
        return a;
    }
    1ll x1, y1;
    1ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}

11 inv_mod(1ll a, 1ll mod){
    1ll x, y;
    1ll g = exgcd(a, mod, x, y);
    assert(abs(g) == 1);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

bool is_prime(1ll n){
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (1ll i = 5; i <= n / i; i += 6){
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

vector<pair<1ll, int>> factorize_pairs(1ll n){
    vector<pair<1ll, int>> ret;
    for (1ll p : {2, 3, 5}){
        if (n % p == 0){
            int cnt = 0;
            while (n % p == 0){
                n /= p;
                cnt++;
            }
            ret.push_back({p, cnt});
        }
    }
    static int inc[] = {4, 2, 4, 2, 4, 6, 2, 6};
    for (1ll p = 7, i = 0; p <= n / p; p += inc[i++ & 7]){
        if (n % p == 0){
            int cnt = 0;
            while (n % p == 0){
                n /= p;
                cnt++;
            }
            ret.push_back({p, cnt});
        }
    }
    if (n > 1) ret.push_back({n, 1});
    return ret;
}

vector<1ll> factorize(1ll n){
    vector<1ll> ret;
    for (auto [p, cnt] : factorize_pairs(n)){
        while (cnt-- > 0) ret.push_back(p);
    }
    return ret;
}

vector<1ll> divisors(1ll n){
    vector<1ll> ret;
    for (1ll i = 1; i <= n / i; i++){
        if (n % i == 0){
            ret.push_back(i);
            if (i != n / i) ret.push_back(n / i);
        }
    }
    return ret;
}

11 phi_single(1ll n){
    1ll ret = n;
    for (auto [p, cnt] : factorize_pairs(n)){
        ret -= ret / p;
    }
    return ret;
}

11 num_divisors(1ll n){
    1ll ret = 1;
    for (auto [p, cnt] : factorize_pairs(n)){
        ret *= cnt + 1;
    }
    return ret;
}

11 sum_divisors(1ll n){
    1ll ret = 1;
    for (auto [p, cnt] : factorize_pairs(n)){
        1ll cur = 1, pw = 1;
        for (int i = 0; i < cnt; i++){

```

```

        pw *= p;
        cur += pw;
    }
    ret += cur;
}

vector<int> spf, primes;
void sieve_spf(int n){
    spf.assign(n + 1, 0);
    primes.clear();
    if (n >= 1) spf[1] = 1;
    for (int i = 2; i <= n; i++){
        if (!spf[i]){
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes){
            1ll x = 1LL * i * p;
            if (x > n) break;
            spf[x] = p;
            if (p == spf[i]) break;
        }
    }
}

vector<int> phi;
void sieve_phi(int n){
    phi.resize(n + 1);
    iota(phi.begin(), phi.end(), 0);
    for (int i = 2; i <= n; i++){
        if (phi[i] == i){
            for (int j = i; j <= n; j += i){
                phi[j] -= phi[j] / i;
            }
        }
    }
}

vector<int> mobius;
void sieve_mobius(int n){
    mobius.assign(n + 1, 0);
    vector<int> lp(n + 1, ps);
    if (n >= 1) mobius[1] = 1;
    for (int i = 2; i <= n; i++){
        if (!lp[i]){
            lp[i] = i;
            ps.push_back(i);
            mobius[i] = -1;
        }
        for (int p : ps){
            1ll x = 1LL * i * p;
            if (x > n) break;
            lp[x] = p;
            if (p == lp[i]){
                mobius[x] = 0;
                break;
            }
            mobius[x] = -mobius[i];
        }
    }
}

vector<vector<int>> divs;
void sieve_divisors(int n){
    divs.assign(n + 1, {});
    for (int i = 1; i <= n; i++){
        for (int j = i; j <= n; j += i){
            divs[j].push_back(i);
        }
    }
}

vector<vector<int>> pfact;
void sieve_prime_factors(int n){
    pfact.assign(n + 1, {});
    for (int i = 2; i <= n; i++){
        if (pfact[i].empty()){
            for (int j = i; j <= n; j += i){
                pfact[j].push_back(i);
            }
        }
    }
}

void SOLVE(){
}

```

Pollard_Rho_Miller_Rabin.cpp

```

using u64 = uint64_t;
using u128 = __uint128_t;

u64 mul_mod(u64 a, u64 b, u64 mod){
    return (u128)a * b % mod;
}

u64 add_mod(u64 a, u64 b, u64 mod){
    a += b;
    if (a >= mod || a < 0) a -= mod;
    return a;
}

u64 fp(u64 a, u64 b, u64 mod){
    u64 res = 1 % mod;
    while (b){
        if (b & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        b >>= 1;
    }
    return res;
}

bool miller_rabin(u64 n){

```

```

if (n < 2) return false;
for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}){
    if (n % p == 0) return n == p;
}
u64 d = n - 1, s = 0;
while ((d & 1) == 0){
    d >>= 1;
    s++;
}
for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL,
    ↪ 1795265022ULL}){
    if (a % n == 0) continue;
    u64 x = fp(a, d, n);
    if (x == 1 || x == n - 1) continue;
    bool comp = true;
    for (u64 r = 1; r < s; r++){
        x = mul_mod(x, x, n);
        if (x == n - 1){
            comp = false;
            break;
        }
    }
    if (comp) return false;
}
return true;
}
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
u64 pollard_rho(u64 n){
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    while (true){
        u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
        u64 y = uniform_int_distribution<u64>(0, n - 1)(rng);
        u64 x = 0, ys = 0, q = 1, g = 1;
        auto f = [&](u64 v){ return add_mod(mul_mod(v, v, n), c, n); };
        for (u64 r = 1; g == 1; r <= 1){
            x = y;
            for (u64 i = 0; i < r; i++) y = f(y);
            for (u64 k = 0; k < r && g == 1; k += 128){
                ys = y;
                for (u64 i = 0; i < min<u64>(128, r - k); i++){
                    y = f(y);
                    u64 dif = x > y ? x - y : y - x;
                    if (dif) q = mul_mod(q, dif, n);
                }
                g = gcd(q, n);
            }
        }
        if (g == n){
            do{
                ys = f(ys);
                u64 dif = x > ys ? x - ys : ys - x;
                g = gcd(dif, n);
            }while (g == 1);
        }
        if (g != n) return g;
    }
}
void factor(u64 n, vector<u64> &ret){
    if (n == 1) return;
    if (miller_rabin(n)){
        ret.push_back(n);
        return;
    }
    u64 d = pollard_rho(n);
    factor(d, ret);
    factor(n / d, ret);
}
vector<u64> factorize(u64 n){
    vector<u64> ret;
    factor(n, ret);
    sort(ret.begin(), ret.end());
    return ret;
}
vector<pair<u64, int>> factorize_pairs(u64 n){
    vector<u64> f = factorize(n);
    vector<pair<u64, int>> ret;
    for (auto p : f){
        if (ret.empty() || ret.back().first != p) ret.push_back({p, 1});
        else ret.back().second++;
    }
    return ret;
}
void SOLVE(){
}

```

Primitive root.cpp

```

ll fp(ll a, ll b, ll mod){
    a %= mod;
    ll res = 1 % mod;
    while (b){
        if (b & 1) res = (__int128)res * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return res;
}
// primitive root modulo prime p
ll primitive_root(ll p){
    if (p == 2) return 1;
    ll phi = p - 1, x = phi;
    vector<ll> fact;
    for (ll i = 2; i <= x / i; i++){
        if (x % i == 0){
            fact.push_back(i);
            while (x % i == 0) x /= i;
        }
    }
    if (x > 1) fact.push_back(x);
    for (ll g = 2; ; g++){

```

```

bool ok = true;
for (ll q : fact){
    if (fp(g, phi / q, p) == 1){
        ok = false;
        break;
    }
}
if (ok) return g;
}
}
void SOLVE(){
}

```

rand.cpp

```

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());

int rnd(int l, int r){
    return uniform_int_distribution<int>(l, r)(rng);
}
ll rndll(ll l, ll r){
    return uniform_int_distribution<ll>(l, r)(rng64);
}
template<typename T>
void shuffle_vec(vector<T> &v){
    shuffle(v.begin(), v.end(), rng);
}
void SOLVE() {
}

```

Segmented sieve.cpp

```

vector<int> simple_sieve(int n){
    vector<bool> is_prime(n + 1, true);
    vector<int> primes;
    if (n >= 0) is_prime[0] = false;
    if (n >= 1) is_prime[1] = false;
    for (int i = 2; i <= n; i++){
        if (!is_prime[i]) continue;
        primes.push_back(i);
        if (1LL * i * i <= n){
            for (ll j = 1LL * i * i; j <= n; j += i){
                is_prime[j] = false;
            }
        }
    }
    return primes;
}
vector<bool> segmented_sieve_mask(ll l, ll r){
    if (l > r) return {};
    int root = sqrtl(r);
    while (1LL * (root + 1) * (root + 1) <= r) root++;
    while (1LL * root * root > r) root--;
    vector<int> primes = simple_sieve(root);
    vector<bool> is_prime(r - l + 1, true);
    for (ll p : primes){
        ll start = max(p * p, ((l + p - 1) / p) * p);
        for (ll j = start; j <= r; j += p){
            is_prime[j - l] = false;
        }
    }
    for (ll x = l; x <= min(r, 1LL); x++){
        is_prime[x - l] = false;
    }
    return is_prime;
}
// returns all primes in [l, r]
vector<ll> segmented_sieve(ll l, ll r){
    vector<bool> is_prime = segmented_sieve_mask(l, r);
    vector<ll> primes;
    for (ll i = 0; i < (ll)is_prime.size(); i++){
        if (is_prime[i]) primes.push_back(l + i);
    }
    return primes;
}
void SOLVE(){
}

```

SOS dp.cpp

```

bool is_power_two(int n){
    return n > 0 && (n & (n - 1)) == 0;
}
template<typename T>
void subset_zeta(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask] += a[mask ^ bit];
        }
    }
}
template<typename T>
void subset_mobius(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask] -= a[mask ^ bit];
        }
    }
}
template<typename T>

```

```

void superset_zeta(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask ^ bit] += a[mask];
        }
    }
}

template<typename T>
void superset_mobius(vector<T> &a){
    int n = a.size();
    assert(is_power_two(n));
    for (int bit = 1; bit < n; bit <= 1){
        for (int mask = 0; mask < n; mask++){
            if (mask & bit) a[mask ^ bit] -= a[mask];
        }
    }
}

vector<int> spf, primes;
void sieve_primes(int n){
    spf.assign(n + 1, 0);
    primes.clear();
    if (n >= 1) spf[1] = 1;
    for (int i = 2; i <= n; i++){
        if (!spf[i]){
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes){
            ll x = 1LL * i * p;
            if (x > n) break;
            spf[x] = p;
            if (p == spf[i]) break;
        }
    }
}

void ensure_primes(int n){
    if ((int)spf.size() <= n) sieve_primes(n);
}

template<typename T>
void divisor_zeta(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = 1; i * p <= n; i++){
            a[i * p] += a[i];
        }
    }
}

template<typename T>
void divisor_mobius(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = n / p; i >= 1; i--){
            a[i * p] -= a[i];
        }
    }
}

template<typename T>
void multiple_zeta(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = n / p; i >= 1; i--){
            a[i] += a[i * p];
        }
    }
}

template<typename T>
void multiple_mobius(vector<T> &a){
    int n = (int)a.size() - 1;
    ensure_primes(n);
    for (int p : primes){
        if (p > n) break;
        for (int i = 1; i * p <= n; i++){
            a[i] -= a[i * p];
        }
    }
}

template<typename T>
vector<T> gcd_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    multiple_zeta(a);
    multiple_zeta(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    multiple_mobius(a);
    return a;
}

template<typename T>
vector<T> lcm_convolution(vector<T> a, vector<T> b){
    assert(a.size() == b.size());
    divisor_zeta(a);
    divisor_zeta(b);
    for (int i = 0; i < (int)a.size(); i++) a[i] *= b[i];
    divisor_mobius(a);
    return a;
}

void SOLVE(){
}

```

xor basis.cpp

```

template<int LG = 60>
struct XorBasis{

```

```

array<ll, LG> basis{};
vector<ll> rref;
ll cnt = 0;
int sz = 0;

void clear(){
    basis.fill(0);
    rref.clear();
    cnt = 0;
    sz = 0;
}

bool insert(ll x, bool count = true){
    if (count) cnt++;
    for (int i = LG - 1; i >= 0; i--){
        if (!(x >> i & 1)) continue;
        if (!basis[i]){
            basis[i] = x;
            sz++;
            rref.clear();
            return true;
        }
        x ^= basis[i];
    }
    return false;
}

void merge(const XorBasis &o){
    cnt += o.cnt;
    for (int i = 0; i < LG; i++){
        if (o.basis[i]) insert(o.basis[i], false);
    }
}

void anding(ll val){
    vector<ll> v;
    for (int i = 0; i < LG; i++){
        if (basis[i] & val) v.push_back(basis[i] & val);
    }
    basis.fill(0);
    rref.clear();
    sz = 0;
    for (ll x : v) insert(x, false);
}

int rank() const {
    return sz;
}

ll count_distinct() const {
    return sz >= 63 ? LLONG_MAX : (1LL << sz);
}

ll count_zero_subsets() const {
    return cnt - sz >= 63 ? LLONG_MAX : (1LL << (cnt - sz));
}

bool can(ll x) const {
    for (int i = LG - 1; i >= 0; i--){
        if (x >> i & 1){
            if (!basis[i]) return false;
            x ^= basis[i];
        }
    }
    return true;
}

ll max_xor(ll x = 0) const {
    for (int i = LG - 1; i >= 0; i--){
        if (basis[i] && (x ^ basis[i]) > x) x ^= basis[i];
    }
    return x;
}

ll min_xor(ll x) const {
    for (int i = LG - 1; i >= 0; i--){
        if (basis[i] && (x ^ basis[i]) < x) x ^= basis[i];
    }
    return x;
}

ll min_xor() const {
    if (sz == 0) return 0;
    XorBasis tmp = *this;
    tmp.normalize();
    ll ans = LLONG_MAX;
    for (ll x : tmp.rref){
        ans = min(ans, x);
    }
    return ans;
}

void normalize(){
    rref.clear();
    for (int i = 0; i < LG; i++){
        if (!basis[i]) continue;
        ll x = basis[i];
        for (int j = i - 1; j >= 0; j--){
            if (x >> j & 1) x ^= basis[j];
        }
        rref.push_back(x);
    }
    sort(rref.rbegin(), rref.rend());
}

ll kth(ll k){
    if (rref.empty() && sz) normalize();
    if (sz >= 63 || k >= (1LL << sz)) return -1;
    ll ans = 0;
    for (int i = 0; i < sz; i++){
        if (k >> (sz - i - 1) & 1) ans ^= rref[i];
    }
    return ans;
}

ll count_le(ll val){
    if (val < 0) return 0;
    if (rref.empty() && sz) normalize();
    if (LG < 63 && val >= (1LL << LG)) return sz >= 63 ? LLONG_MAX : (1LL << sz);
    ll ans = 0, mask = 0;
    int j = 0;
    for (int b = LG - 1; b >= 0; b--){
        bool piv = j < sz && (63 - __builtin_clzll(rref[j])) == b;
        int bit = (val >> b) & 1;
        if (piv){

```

```

        if (bit){
            if (sz - 1 - j >= 63) return LLONG_MAX;
            ans += 1LL << (sz - 1 - j);
            mask ^= rref[j];
        }
        j++;
    }else{
        int cur = (mask >> b) & 1;
        if (cur < bit){
            if (sz - j >= 63) return LLONG_MAX;
            return ans + (1LL << (sz - j));
        }
        if (cur > bit) return ans;
    }
}
return ans + 1;
}
};

void SOLVE(){
}

```

5 strings

Aho.cpp

```
struct AC{
    int N, P;
    static const int A = 26;
    vector<array<int, A>> next;
    vector<int> link, out_link, pat_len;
    vector<vector<int>> out, rev;
    AC(): N(0), P(0) {node();}
    int node() {
        next.push_back(array<int, A>{0});
        rev.push_back({});
        link.emplace_back(0);
        out_link.emplace_back(0);
        out.emplace_back();
        return N++;
    }
    inline int get(char c){ return c - 'a'; }
    int add_pattern(const string & T){
        int u = 0;
        for(auto c : T){
            if(!next[u][get(c)]) next[u][get(c)] = node();
            u = next[u][get(c)];
        }
        out[u].push_back(P);
        pat_len.push_back(T.length());
        return P++;
    }
    void build(){
        queue<int> q; q.push(0);
        while(!q.empty()){
            int u = q.front(); q.pop();
            for(int c = 0; c < A; c++){
                int v = next[u][c];
                if(!v) next[u][c] = next[link[u]][c];
                else{
                    link[v] = u ? next[link[u]][c] : 0;
                    rev[link[v]].push_back(v);
                    out_link[v] = out[link[v]].empty() ? out_link[link[v]] :
                        out_link[v];
                    q.push(v);
                }
            }
        }
    }
    int advance(int u, char c){
        return next[u][get(c)];
    }
    vector<vector<int>> search(const string & s){
        int n = (int)s.length(), m = (int)pat_len.size();
        int u = 0;
        vector<vector<int>> ret(m);
        for(int i = 0; i < n; i++){
            char c = s[i];
            u = advance(u, c);
            int x = u;
            while(x != 0){
                for(int v : out[x]) ret[v].push_back(i - pat_len[v] + 1);
                x = out_link[x];
            }
        }
        return ret;
    }
};

void SOLVE() {
}
```

dynamic_aho.cpp

```
struct AC{
    int N, P;
    static const int A = 26;
    vector<array<int, A>> next;
    vector<int> link, out_link, cnt;
    vector<vector<int>> out, rev;
    AC(): N(0), P(0) {node();}
    int node() {
        next.push_back(array<int, A>{0});
        rev.push_back({});
        cnt.emplace_back(0);
        link.emplace_back(0);
        out_link.emplace_back(0);
        out.emplace_back();
        return N++;
    }
    inline int get(char c){ return c - 'a'; }
    int add_pattern(const string & T){
        int u = 0;
        for(auto c : T){
            if(!next[u][get(c)]) next[u][get(c)] = node();
            u = next[u][get(c)];
        }
        out[u].push_back(P);
        cnt[u]++;
        return P++;
    }
    void build(){
        queue<int> q; q.push(0);
        while(!q.empty()){
            int u = q.front(); q.pop();
            for(int c = 0; c < A; c++){
                int v = next[u][c];
                if(!v) next[u][c] = next[link[u]][c];
                else{
                    link[v] = u ? next[link[u]][c] : 0;
                    rev[link[v]].push_back(v);
                    cnt[v] += cnt[link[v]];
                    out_link[v] = out[link[v]].empty() ? out_link[link[v]] :
                        out_link[v];
                    link[v];
                }
            }
        }
    }
};
```

```
        q.push(v);
    }
}

int advance(int u, char c){
    return next[u][get(c)];
}

vector<vector<int>> search(const string & s, vector<int>& pat){
    int n = (int)s.length(), m = (int)pat.size();
    int u = 0;
    vector<vector<int>> ret(m);
    for(int i = 0; i < n; i++){
        char c = s[i];
        u = advance(u, c);
        int x = u;
        while(x != 0){
            for(int v : out[x]) ret[v].push_back(i - pat[v] + 1);
            x = out_link[x];
        }
    }
    return ret;
}

struct dynamic_AC{
    int lg;
    vector<vector<string>> a;
    vector<AC> aho;
    dynamic_AC(int LOG = 20){
        lg = LOG;
        a = vector<vector<string>>(lg);
        aho = vector<AC>(lg);
    }

    void insert(const string & s){
        vector<string> have = {s};
        for(int i = 0; i < lg; i++){
            if(i == lg - 1){
                lg++;
                a.push_back({});
                aho.push_back(AC());
            }
            if(a[i].empty()){
                swap(have, a[i]);
                for(auto & v : a[i]) aho[i].add_pattern(v);
                aho[i].build();
                break;
            }
            have.insert(end(have), begin(a[i]), end(a[i]));
            vector<string>().swap(a[i]);
            aho[i] = AC();
        }
    }

    ll cnt(const string & s){
        ll answer = 0;
        for(int i = 0; i < lg; i++){
            int state = 0;
            for(auto & v : s){
                state = aho[i].advance(state, v);
                answer += aho[i].cnt[state];
            }
        }
        return answer;
    }
};

void SOLVE() {
}
```

Generalized suffix automaton.cpp

```
struct GSAM{
    static const int A = 26;
    struct node{
        int len, link;
        bool is_cloned;
        int nxt[A];
        node(){
            memset(nxt, -1, sizeof nxt);
            len = 0;
            link = -1;
            is_cloned = false;
        }
    };
    int sz;
    vector<node> t;
    vector<int> cnt;
    vector<vector<int>> inv, g;
    vector<ll> freq;
    GSAM(){
        GSAM(int n, int m){
            init(n, m);
        }
        void init(int n, int m){
            t.assign(2 * n + 5, node());
            cnt.assign(2 * n + 5, 0);
            freq.assign(2 * n + 5, -1);
            g.assign(2 * n + 5, {});
            inv.assign(m, {});
            sz = 1;
        }
        int get(char c){
            return c - 'a';
        }
        int extend(char c, int last){
            int c_idx = get(c);
            if (t[last].nxt[c_idx] != -1){
                int q = t[last].nxt[c_idx];
                if (t[last].len + 1 == t[q].len) return q;
            }
        }
    };
};
```

```

    int clone = sz++;
    t[clone] = t[q];
    t[clone].len = t[last].len + 1;
    t[clone].is_cloned = true;
    int p = last;
    while (p != -1 && t[p].nxt[c_idx] == q){
        t[p].nxt[c_idx] = clone;
        p = t[p].link;
    }
    t[q].link = clone;
    return clone;
}
int cur = sz++;
t[cur].len = t[last].len + 1;
int p = last;
while (p != -1 && t[p].nxt[c_idx] == -1){
    t[p].nxt[c_idx] = cur;
    p = t[p].link;
}
if (p == -1){
    t[cur].link = 0;
}else{
    int q = t[p].nxt[c_idx];
    if (t[p].len + 1 == t[q].len){
        t[cur].link = q;
    }else{
        int clone = sz++;
        t[clone] = t[q];
        t[clone].len = t[p].len + 1;
        while (p != -1 && t[p].nxt[c_idx] == q){
            t[p].nxt[c_idx] = clone;
            p = t[p].link;
        }
        t[q].link = t[cur].link = clone;
        t[clone].is_cloned = true;
    }
}
return cur;
}
void build_tree(){
    for (int i = 1; i < sz; i++){
        g[t[i].link].push_back(i);
    }
}
void build(const vector<string> &strings){
    for (int i = 0; i < (int)strings.size(); i++){
        int lst = 0;
        inv[i].resize(strings[i].size());
        for (int j = 0; j < (int)strings[i].size(); j++){
            lst = extend(strings[i][j], lst);
            cnt[lst]++;
            inv[i][j] = lst;
        }
    }
    build_tree();
}
ll occ(int i){
    if (~freq[i]) return freq[i];
    ll ret = cnt[i];
    for (auto x : g[i]){
        ret += occ(x);
    }
    return freq[i] = ret;
}
void getMask(int u, vector<ll> &mask){
    for (int v : g[u]){
        getMask(v, mask);
        mask[u] |= mask[v];
    }
}
int get_lcs(int K){
    vector<ll> mask(sz, 0);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            mask[inv[i][j]] |= (1LL << i);
        }
    }
    getMask(0, mask);
    int max_len = 0;
    ll target = (1LL << K) - 1;
    for (int i = 1; i < sz; i++){
        if ((mask[i] & target) == target){
            max_len = max(max_len, t[i].len);
        }
    }
    return max_len;
}
ll distinct_substrings(){
    ll total = 0;
    for (int i = 1; i < sz; i++){
        total += t[i].len - t[t[i].link].len;
    }
    return total;
}
ll match(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    return occ(u);
}
ll calc(int u, vector<ll> &dp){
    if (dp[u] != -1) return dp[u];
    dp[u] = 1;
    for (int c = 0; c < A; c++){
        if (t[u].nxt[c] != -1){
            dp[u] += calc(t[u].nxt[c], dp);
        }
    }
    return dp[u];
}
string get_kth(ll K){

```

```

    vector<ll> dp(sz, -1);
    calc(0, dp);
    string res = "";
    int u = 0;
    while (K > 0){
        for (int c = 0; c < A; c++){
            if (t[u].nxt[c] != -1){
                int v = t[u].nxt[c];
                if (dp[v] >= K){
                    res += (char)('a' + c);
                    u = v;
                    K--;
                    break;
                }else{
                    K -= dp[v];
                }
            }
        }
    }
    return res;
}
int shortest_distinguishing(){
    vector<ll> mask(sz, 0);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            mask[inv[i][j]] |= (1LL << i);
        }
    }
    getMask(0, mask);
    int min_len = 1e9;
    for (int i = 1; i < sz; i++){
        if (__builtin_popcountll(mask[i]) == 1){
            min_len = min(min_len, t[t[i].link].len + 1);
        }
    }
    return min_len == 1e9 ? -1 : min_len;
}
vector<pair<int, int>> get_all_occurrences(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return {};
        u = t[u].nxt[get(c)];
    }
    vector<vector<pair<int, int>>> state_endpoints(sz);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            state_endpoints[inv[i][j]].push_back({i, j});
        }
    }
    vector<pair<int, int>> res;
    collect(u, (int)P.length(), state_endpoints, res);
    return res;
}
void collect(int u, int len, vector<vector<pair<int, int>>> &ends,
    ⇐ vector<pair<int, int>> &res){
    for (auto ep : ends[u]){
        res.push_back({ep.first, ep.second - len + 1});
    }
    for (int v : g[u]){
        collect(v, len, ends, res);
    }
}
ll dfsSp(int u, vector<int> &cnt2, vector<ll> &freq2){
    if (freq2[u] != -1) return freq2[u];
    ll ret = cnt2[u];
    for (int v : g[u]){
        ret += dfsSp(v, cnt2, freq2);
    }
    return freq2[u] = ret;
}
ll string_specific_occ(const string &P, int str_idx){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    vector<int> cnt2(sz, 0);
    for (int j = 0; j < (int)inv[str_idx].size(); j++){
        cnt2[inv[str_idx][j]]++;
    }
    vector<ll> freq2(sz, -1);
    return dfsSp(u, cnt2, freq2);
}
};

void SOLVE(){
}

```

KMP.cpp

```

vector<int> KMP(const string &s){
    int n = s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++){
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]){
            j = pi[j - 1];
        }
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
vector<int> find_occurrences(const string &pat, const string &text){
    string s = pat + '#' + text;
    vector<int> pi = KMP(s);
    int m = pat.size();
    for (int i = m + 1; i < (int)s.size(); i++){
        if (pi[i] == m) ret.push_back(i - 2 * m);
    }
    return ret;
}

```

```

vector<vector<int>> automaton(string s){ // aut[s.length() + 1][26]
    s += '#';
    int n = s.length();
    vector<int> pi = KMP(s);
    vector<vector<int>> aut(n, vector<int>(26));
    for (int i = 0; i < n; i++){
        for (int j = 0; j < 26; j++){
            if (i > 0 && 'a' + j != s[i]){
                aut[i][j] = aut[pi[i - 1]][j];
            }else{
                aut[i][j] = i + ('a' + j == s[i]);
            }
        }
    }
    return aut;
}

void SOLVE(){
}

```

manacher.cpp

```

struct manacher{
    vector<int> p, odd, even;
    vector<ll> starts, ends;
    manacher(){
        manacher(const string &s){
            build(s);
        }
        void run_manacher(const string &s){
            int n = s.length();
            p.assign(n, 1);
            int l = 1, r = 1;
            for (int i = 1; i < n; i++){
                p[i] = i < r ? min(r - i, p[l + r - i]) : 1;
                while (i - p[i] >= 0 && i + p[i] < n && s[i - p[i]] == s[i + p[i]]){
                    p[i]++;
                }
                if (r < i + p[i]){
                    r = i + p[i];
                    l = i - p[i];
                }
            }
        }
        void build(const string &s){
            string t = "#";
            t.reserve(s.length() * 2 + 1);
            for (char c : s){
                t += c;
                t += '#';
            }
            run_manacher(t);
            build_radix(s.length());
            build_counts(s.length());
        }
        void build_radix(int n){
            odd.assign(n, 0);
            even.assign(n, 0);
            for (int i = 0; i < n; i++){
                odd[i] = get_longest(i, false) / 2 + 1;
                even[i] = get_longest(i, true) / 2;
            }
        }
        int get_longest(int cen, bool even){
            return p[2 * cen + 1 + even] - 1;
        }
        bool ispal(int l, int r){
            return (r - l + 1) <= get_longest((l + r) >> 1, 1 % 2 != r % 2);
        }
        pair<int, int> longest_palindrome(){
            int best = 0, pos = 0;
            for (int i = 0; i < (int)p.size(); i++){
                if (p[i] - 1 > best){
                    best = p[i] - 1;
                    pos = i;
                }
            }
            int l = (pos - best) / 2;
            return {l, l + best - 1};
        }
        ll count_palindromes(){
            ll ret = 0;
            for (int x : p){
                ret += x / 2;
            }
            return ret;
        }
        void build_counts(int n){
            starts.assign(n + 1, 0);
            ends.assign(n + 1, 0);
            for (int i = 0; i < (int)p.size(); i++){
                int len = p[i] - 1;
                if (!len) continue;
                int l = (i - len + 1) / 2;
                int r = (i - 1) / 2;
                starts[l]++;
                starts[r + 1]--;
                int el = i - r - 1;
                int er = i - l - 1;
                ends[el]++;
                ends[er + 1]--;
            }
            for (int i = 1; i < n; i++){
                starts[i] += starts[i - 1];
                ends[i] += ends[i - 1];
            }
            starts.pop_back();
            ends.pop_back();
        }
};

void SOLVE(){
}

```

```

}

```

palindromic tree.cpp

```

struct PalindromicTree{
    static const int A = 26;
    struct node{
        int nxt[A], len, st, en, link, cnt, oc, smart_link[A];
        node(){
            memset(nxt, 0, sizeof nxt);
            for (int i = 0; i < A; i++) smart_link[i] = 1;
            oc = cnt = 0;
        }
    };
    string s;
    vector<node> t;
    vector<array<int, 4>> changes;
    int sz, last;
    vector<int> pref;
    ll tot_pals;
    vector<ll> tot_pals_hist;
    PalindromicTree(){
        init();
    }
    void init(){
        s = "";
        t.assign(3, node());
        sz = 2;
        last = 2;
        t[1].len = -1;
        t[1].link = 1;
        t[2].len = 0;
        t[2].link = 1;
        changes.clear();
        pref.clear();
        tot_pals = 0;
        tot_pals_hist.clear();
    }
    void build(const string &str){
        init();
        for (char c : str){
            extend(c);
        }
    }
    int extend(char c){
        tot_pals_hist.push_back(tot_pals);
        int cur = last, pos = s.size();
        pref.push_back(0);
        if (pos) pref[pos] = pref[pos - 1];
        s += c;
        int ch = c - 'a';
        if (pos - t[cur].len - 1 < 0 || s[pos - t[cur].len - 1] != c){
            cur = t[cur].smart_link[ch];
        }
        if (t[cur].nxt[ch]){
            changes.push_back({last, t[cur].nxt[ch], -1, -1});
            last = t[cur].nxt[ch];
            t[last].oc++;
            tot_pals += t[last].cnt;
            return 0;
        }
        changes.push_back({last, -1, cur, ch});
        sz++;
        last = sz;
        t.emplace_back();
        t[sz].oc = 1;
        t[sz].len = t[cur].len + 2;
        pref[pos] = max(pref[pos], t[sz].len);
        t[cur].nxt[ch] = sz;
        t[sz].en = pos;
        t[sz].st = pos - t[sz].len + 1;
        if (t[sz].len == 1){
            t[sz].link = 2;
            t[sz].cnt = 1;
            t[sz].smart_link[ch] = 2;
            tot_pals += t[sz].cnt;
            return 1;
        }
        t[sz].link = t[t[cur].smart_link[ch]].nxt[ch];
        for (int i = 0; i < A; i++){
            int x = t[sz].link;
            if (pos - t[x].len >= 0 && s[pos - t[x].len] - 'a' == i){
                t[sz].smart_link[i] = x;
            }else{
                t[sz].smart_link[i] = t[x].smart_link[i];
            }
        }
        t[sz].cnt = 1 + t[t[sz].link].cnt;
        tot_pals += t[sz].cnt;
        return 1;
    }
    void rollback(){
        if (s.empty()) return;
        s.pop_back();
        pref.pop_back();
        auto x = changes.back();
        int prvlst = x[0], oc = x[1], c = x[2], ch = x[3];
        changes.pop_back();
        last = prvlst;
        if (oc == -1){
            t[c].nxt[ch] = 0;
            t.pop_back();
            sz--;
        }else{
            t[oc].oc--;
        }
        tot_pals = tot_pals_hist.back();
        tot_pals_hist.pop_back();
    }
    int cnt_dis() const{ return sz - 2; }
    ll pal_cnt() const{ return tot_pals; }
    void propagate_occurrences(){
}

```

```

        for (int i = sz; i >= 3; i--){
            t[t[i].link].oc += t[i].oc;
        }
    }
    string get_longest() const{
        int max_len = 0, best_st = -1;
        for (int i = 3; i <= sz; i++){
            if (t[i].len > max_len){
                max_len = t[i].len;
                best_st = t[i].st;
            }
        }
        if (best_st == -1) return "";
        return s.substr(best_st, max_len);
    }
    vector<pair<string, int>> get_all() const{
        vector<pair<string, int>> res;
        for (int i = 3; i <= sz; i++){
            res.push_back({s.substr(t[i].st, t[i].len), t[i].oc});
        }
        return res;
    }
};

void SOLVE(){
}

```

String_Hashing.cpp

```

using u64 = uint64_t;

const int N = 2e5 + 5;
const u64 md = (1ULL << 61) - 1;
const u64 B = 788294215192145664ULL, IB = 285196104568472234ULL;
inline u64 add(u64 a, u64 b){
    return a + b >= md ? a + b - md : a + b;
}
inline u64 mul(u64 a, u64 b){
    __uint128_t l = (__uint128_t)a * b;
    u64 ret = (l & md) + (l >> 61);
    return ret >= md ? ret - md : ret;
}
vector<u64> PB(N), PIB(N);
void init_hash(){
    PB[0] = PIB[0] = 1;
    for (int i = 1; i < N; i++){
        PB[i] = mul(PB[i - 1], B);
        PIB[i] = mul(PIB[i - 1], IB);
    }
}
struct Hash : pair<u64, int>{
    Hash(){
        Hash(const pair<u64, int> &x) : pair(x){}
        Hash(int x) : pair(x, 1){}
        Hash(const string &x){
            for (char c : x) *this = *this + c;
        }
        friend Hash operator+(const Hash &f, const Hash &o){
            return pair{add(mul(f.first, PB[o.second]), o.first), f.second +
                o.second};
        }
};
struct Hashing{
    vector<Hash> p, rev;
    Hashing(){
        Hashing(const string &t){
            build(t);
        }
        void build(const string &t){
            clear();
            for (char c : t) push_back(c);
        }
        void push_back(char c){
            if (p.empty()){
                p.push_back(Hash(c));
                rev.push_back(Hash(c));
            }
            else{
                p.push_back(p.back() + Hash(c));
                rev.push_back(Hash(c) + rev.back());
            }
        }
        void pop_back(){
            assert(!p.empty());
            p.pop_back();
            rev.pop_back();
        }
        Hash get(int l, int r){
            if (l > r) return {};
            if (!l) return p[r];
            return pair{add(mul(p[r].first, md - mul(p[l - 1].first, PB[r - l + 1])), r
                - l + 1);
        }
        Hash inv(int l, int r){
            if (l > r) return {};
            if (!l) return rev[r];
            u64 val = add(rev[r].first, md - rev[l - 1].first);
            val = mul(val, PIB[l]);
            return pair{val, r - l + 1};
        }
        bool ispal(int l, int r){
            return get(l, r) == inv(l, r);
        }
        int size() const{
            return p.size();
        }
        void clear(){
            p.clear();
            rev.clear();
        }
};

void SOLVE(){
}

```

```

}

```

Suffixautomaton.cpp

```

struct SAM{
    static const int A = 26;
    struct node{
        int len, link, firstpos, lstpos;
        bool is_cloned;
        int nxt[A];
    }
    node(){
        memset(nxt, -1, sizeof nxt);
        len = 0;
        link = -1;
        firstpos = 0;
        lstpos = 0;
        is_cloned = false;
    }
};

int sz, last;
vector<node> t;
vector<int> terminal, inv;
vector<ll> freq;
vector<vector<int>> g;
SAM(){
    SAM(int n){
        init(n);
    }
    void init(int n){
        t.assign(2 * n + 5, node());
        terminal.assign(2 * n + 5, 0);
        inv.assign(n + 5, 0);
        freq.assign(2 * n + 5, -1);
        g.assign(2 * n + 5, {});
        sz = 1;
        last = 0;
    }
    int get(char c){
        return c - 'a';
    }
    void extend(char c){
        int cur = sz++;
        t[cur].len = t[last].len + 1;
        t[cur].firstpos = t[cur].lstpos = t[cur].len - 1;
        int p = last;
        while (p != -1 && t[p].nxt[get(c)] == -1){
            t[p].nxt[get(c)] = cur;
            p = t[p].link;
        }
        if (p == -1){
            t[cur].link = 0;
        }
        else{
            int q = t[p].nxt[get(c)];
            if (t[p].len + 1 == t[q].len){
                t[cur].link = q;
            }
            else{
                int clone = sz++;
                t[clone] = t[q];
                t[clone].len = t[p].len + 1;
                while (p != -1 && t[p].nxt[get(c)] == q){
                    t[p].nxt[get(c)] = clone;
                    p = t[p].link;
                }
                t[q].link = t[cur].link = clone;
                t[clone].is_cloned = true;
            }
        }
        last = cur;
    }
    void build_tree(){
        for (int i = 1; i < sz; i++){
            g[t[i].link].push_back(i);
        }
    }
    void build(const string &s){
        init(s.size());
        int j = 0;
        for (auto x : s){
            extend(x);
            terminal[last] = j + 1;
            inv[j++] = last;
        }
        build_tree();
    }
    ll occ(int i){
        if (!freq[i]) return freq[i];
        ll ret = terminal[i] > 0;
        for (auto x : g[i]){
            ret += occ(x);
        }
        return freq[i] = ret;
    }
    ll distinct_substrings(){
        ll total = 0;
        for (int i = 1; i < sz; i++){
            total += t[i].len - t[t[i].link].len;
        }
        return total;
    }
    ll calc(int u, vector<ll> &dp){
        if (dp[u] != -1) return dp[u];
        dp[u] = 1;
        for (int c = 0; c < A; c++){
            if (t[u].nxt[c] != -1){
                dp[u] += calc(t[u].nxt[c], dp);
            }
        }
        return dp[u];
    }
    string get_kth(ll K){
        vector<ll> dp(sz, -1);
        calc(0, dp);
    }
}

```

```

if (K <= 0 || K >= dp[0]) return "";
string res = "";
int u = 0;
while (K > 0){
    for (int c = 0; c < A; c++){
        if (t[u].nxt[c] == -1) continue;
        int v = t[u].nxt[c];
        if (dp[v] >= K){
            res += (char)('a' + c);
            u = v;
            K--;
            break;
        }
        K -= dp[v];
    }
}
return res;
}
ll calcAll(int u, vector<ll> &dp){
    if (dp[u] != -1) return dp[u];
    dp[u] = occ(u);
    for (int c = 0; c < A; c++){
        if (t[u].nxt[c] != -1){
            dp[u] += calcAll(t[u].nxt[c], dp);
        }
    }
    return dp[u];
}
string get_kth_all(ll K){
    vector<ll> dp(sz, -1);
    calcAll(0, dp);
    if (K <= 0 || K > dp[0]) return "";
    string res = "";
    int u = 0;
    while (K > 0){
        bool moved = false;
        for (int c = 0; c < A; c++){
            if (t[u].nxt[c] == -1) continue;
            int v = t[u].nxt[c];
            if (dp[v] >= K){
                res += (char)('a' + c);
                u = v;
                K -= occ(v);
                moved = true;
                break;
            }
            K -= dp[v];
        }
        if (!moved || K <= 0) break;
    }
    return res;
}
ll match(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    return occ(u);
}
int first_occurrence(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return -1;
        u = t[u].nxt[get(c)];
    }
    return t[u].firstpos - (int)P.length() + 1;
}
void getOcc(int u, int len, vector<int> &res){
    if (terminal[u] > 0){
        res.push_back(terminal[u] - len);
    }
    for (int v : g[u]){
        getOcc(v, len, res);
    }
}
vector<int> all_occurrences(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return {};
        u = t[u].nxt[get(c)];
    }
    vector<int> res;
    getOcc(u, (int)P.length(), res);
    sort(res.begin(), res.end());
    return res;
}
int lcs(const string &T){
    int u = 0, l = 0, max_len = 0;
    for (char c : T){
        int idx = get(c);
        while (u && t[u].nxt[idx] == -1){
            u = t[u].link;
            l = t[u].len;
        }
        if (t[u].nxt[idx] != -1){
            u = t[u].nxt[idx];
            l++;
        }
        max_len = max(max_len, l);
    }
    return max_len;
}
};

void SOLVE(){
}

```

Trie_strings.cpp

```

struct Trie{
    vector<array<int, 26>> nxt;

```

```

vector<int> cnt, ending;
Trie(){
    init();
}
void init(){
    nxt.clear();
    cnt.clear();
    ending.clear();
    node();
}
int node(){
    nxt.push_back(array<int, 26>{0});
    cnt.push_back(0);
    ending.push_back(0);
    return (int)nxt.size() - 1;
}
void insert(const string &s){
    int u = 0;
    cnt[u]++;
    for (char c : s){
        int idx = c - 'a';
        if (!nxt[u][idx]) nxt[u][idx] = node();
        u = nxt[u][idx];
        cnt[u]++;
    }
    ending[u]++;
}
bool erase(const string &s){
    if (!count(s)) return false;
    int u = 0;
    cnt[u]--;
    for (char c : s){
        int idx = c - 'a';
        u = nxt[u][idx];
        cnt[u]--;
    }
    ending[u]--;
    return true;
}
int count(const string &s){
    int u = 0;
    for (char c : s){
        int idx = c - 'a';
        if (!nxt[u][idx]) return 0;
        u = nxt[u][idx];
    }
    return ending[u];
}
int count_prefix(const string &s){
    int u = 0;
    for (char c : s){
        int idx = c - 'a';
        if (!nxt[u][idx]) return 0;
        u = nxt[u][idx];
    }
    return cnt[u];
}
bool contains(const string &s){
    return count(s) > 0;
}
void clear(){
    init();
}
};

void SOLVE(){
}

```

Z_algorithm.cpp

```

vector<int> zfunction(const string& s){
    int n = s.size();
    vector<int> z(n);
    for(int i = 1, l = 0, r = 0; i < n; i++){
        if(i < r) z[i] = min(r - i, z[i - l]);
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if(i + z[i] > r){
            r = i + z[i];
            l = i;
        }
    }
    return z;
}

void SOLVE() {
}

```

6 DP Optimizations and Misc

divide and conquer dp.cpp

```
const int N = 1e5 + 5;
const ll inf = 4e18;

int n, k;
ll dp[2][N];
int L = 1, R = 0;
ll cur = 0;

void add(int i){
}
void rem(int i){
}
void move(int l, int r){
    while (R < r) add(++R);
    while (L > l) add(--L);
    while (R > r) rem(R--);
    while (L < l) rem(L++);
}
ll cost(int l, int r){
    move(l, r);
    return cur;
}
void calc(){
    vector<array<int, 4>> st;
    st.push_back({1, n, 1, n});
    while (!st.empty()){
        auto [l, r, optL, optR] = st.back();
        st.pop_back();
        if (l > r) continue;
        int mid = (l + r) >> 1;
        pair<ll, int> best = {inf, optL};
        for (int i = optL; i <= min(optR, mid); i++){
            best = min(best, {dp[0][i - 1] + cost(i, mid), i});
        }
        dp[1][mid] = best.first;
        int opt = best.second;
        st.push_back({l, mid - 1, optL, opt});
        st.push_back({mid + 1, r, opt, optR});
    }
}
void compute(){
    for (int i = 1; i <= n; i++){
        dp[0][i] = inf;
    }
    dp[0][0] = 0;
    for (int layer = 1; layer <= k; layer++){
        for (int i = 0; i <= n; i++){
            dp[1][i] = inf;
        }
        L = 1;
        R = 0;
        cur = 0;
        calc();
        for (int i = 0; i <= n; i++){
            dp[0][i] = dp[1][i];
        }
    }
}
void SOLVE(){
}
```

Custom hashing.cpp

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return (size_t) splitmix64(x + FIXED_RANDOM);
    }
};
struct pair_hash {
    size_t operator()(const pair<ll, ll>& p) const noexcept {
        uint64_t a = (uint64_t) p.first;
        uint64_t b = (uint64_t) p.second;
        uint64_t z = a + 0x9e3779b97f4a7c15ULL + (b << 6) + (b >> 2);
        return (size_t) custom_hash::splitmix64(z);
    }
};
void SOLVE() {
}
```